

Peering into the twilight - a systematic review on visual-based investigation of mesophotic ecosystems - POLAR

Claudia Campanini

2026-05-28

Table of Contents

DATA PREPARATION	2
Load libraries	2
Load data.....	3
Subset for studies encompassing polar latitudinal region	3
Cleaning.....	3
MISSING / ND / NOTAPP VALUES	3
GEOGRAPHICAL DISTRIBUTION	4
Provinces.....	5
Ecoregions.....	6
Realms	7
Export geographical distribution summary tables	8
COUNT COLUMNS WITH COMBO	8
Export summary tables.....	9
COUNT METHODS (PURELY TAXONOMIC STUDIES EXCLUDED)	11
COUNT COLUMNS SPLIT.....	13
COUNT METHODS (PURELY TAXONOMIC STUDIES EXCLUDED)	16
PUBLICATIONS PER DEPTH	19
PLATFORM BY DEPTH.....	21
DIRECT - INDIRECT BY DECADE	37
FIELD & INDICATORS	39
Calculate # of ecological indicators per record	43
SAMPLING UNIT.....	44
Prepare a data frame for sampling unit.....	44

Calculate # of sampling unit per record.....	45
Simplified sampling units.....	46
Quadrat extracted from transect	50
DATA TYPE.....	50
HABIT.....	52
SUBSTRATE	54
BENTHIC POSITION	57
TARGET	59
Target level	59
Detected phyla in community studies	61
Target taxonomic groups not at the community level.....	64
New species described per phyla	67
DATA CROSSING	68
VISUALLY SURVEYED SURFACE	68
Observation	68
Temp replicates & exp.....	69
ROV & sample	69
Molecular techniques in taxonomic and non-taxonomic studies.....	69
SESSION INFORMATION AND REFERENCES.....	70

DATA PREPARATION

Load libraries

```
library(here)
```

```
## here() starts at C:/Users/claud/Documents/COMUNE/STUDIO_LAVORO/Data_analysis/Projects/Mesophotic_systematic_lit_rev_analysis
```

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
## intersect, setdiff, setequal, union
```

```
library(ggplot2)  
library(readxl) # to import excel data sets  
library(magrittr)  
library(stringr) #many data wrangling functions  
library(readxl)
```

Load data

```
polar <- read_excel(here("Extracted_data_analysis", "Input_data", "extracted_raw.xlsx"))
```

Subset for studies encompassing polar latitudinal region

```
polar <- subset(polar, str_detect(polar$latitudinalRegion, "polar"))
```

Cleaning

Clean year columns

```
#keep the year only in the year column (in some rows exact time is specified)  
polar$publicationYear <- stringr::str_remove(polar$publicationYear,  
                                             pattern = "\\-.*")
```

MISSING / ND / NOTAPP VALUES

Create a summary table

```
library(purrr)  
  
##  
## Attaching package: 'purrr'  
  
## The following object is masked from 'package:magrittr':  
##  
## set_names  
  
library(tibble)  
  
# Get all column names  
cols <- names(polar[, -c(1:13)])  
  
# Create a summary list  
summary_list <- map(cols, function(col) {  
  data <- polar[, -c(1:13)][[col]]  
  
  tibble(  
    Column = col,  
    unique = paste(unique(data), collapse = ", ")
```

```

nd      = sum(data == "nd", na.rm = TRUE),
notapp  = sum(data == "notapp", na.rm = TRUE),
assigned = sum(!is.na(data) & data != "nd" & data != "notapp"),
`NA`    = sum(is.na(data)),
total   = sum(data == "nd", na.rm = TRUE) +
          sum(data == "notapp", na.rm = TRUE) +
          sum(!is.na(data) & data != "nd" & data != "notapp")
)
})

# Combine into one dataframe
summary_df <- bind_rows(summary_list)

summary_df

## # A tibble: 72 × 7
##   Column                unique      nd notapp assigned `NA`
##   <chr>                <chr>    <int> <int>    <int> <int>
##   <int>
## 1 meso_direct_indirect indirect, mixed      0      0      30      0
## 2 meso_samplngPlatform rov, hov, dredge+ro...      0      0      30      0
## 3 taxonomic_publication NA              0      0       0     30
## 4 meso_substrate      hard, nd, mixed, so...      2      0      24      4
## 5 meso_habitat        rocky_reef, NA, mix...      1      0      27      2
## 6 meso_protection     nd, no, yes          26      0       4      0
## 7 taxonomicTargetLevel comm, single, speci...      0      0      30      0
## 8 habit               mixed, vagile, sess...      0      0      30      0
## 9 environmentalPosition epi, dem+epi, endo+...      0      0      30      0
## 10 detected_meso_seagrass NA              0      0       0     30
## # i 62 more rows

rm(summary_list, cols)

```

GEOGRAPHICAL DISTRIBUTION

```

location <- polar[,c (
  "recordTitle", "DOI", "publicationYear", "prov_code", "ecoreg_code")]

```

Load Marine Ecoregion of the World data

```
meow <- read.csv(file=here("Extracted_data_analysis", "Input_data", "MEOW.csv"))
```

Provinces

```
prov <- location

prov_lvls <-
  unique(
    unlist(
      str_split(location$prov_code, "\\_")
    )
  )

prov[,prov_lvls] <- sapply(prov_lvls,
  function(y){
    str_count(location$prov_code, y)
  }
)

rm(prov_lvls)
```

Create a column with the sum of provinces per publication

```
prov %<>%
  mutate(
    prov_per_record = rowSums(
      across(
        where(is.numeric)
      )
    )
  )

prov$prov_per_record <- as.factor(prov$prov_per_record)

prov_per_record_df <- prov %>%
  group_by(prov_per_record) %>%
  summarise(n_records = length(prov_per_record)) %>%
  mutate(percentage= round(n_records / sum(n_records) * 100,0))

prov_per_record_df

## # A tibble: 3 × 3
##   prov_per_record n_records percentage
##   <fct>          <int>         <dbl>
## 1 1              21          70
## 2 2               8          27
## 3 3               1           3
```

Create a data frame with number of publication per province

```

prov_tot <-
  prov[, -(length(prov)-1)] %>% # filter out the sum column
  summarise(across(where(is.numeric),
                    list(sum)
                  )
            )

prov_tot <- reshape2::melt(prov_tot, id.vars = NULL)

colnames(prov_tot) <- c("Province", "Count")

prov_tot %<>%
  mutate_at("Province",
            stringr::str_replace, "_1", "")

```

Ecoregions

```

ecoreg <- location
ecoreg_lvls <-
  unique(
    unlist(
      str_split(ecoreg$ecoreg_code, "\\_")
    )
  )

ecoreg_lvls <- ecoreg_lvls[! ecoreg_lvls %in% c("")]

ecoreg[, ecoreg_lvls] <- sapply(ecoreg_lvls,
                                function(y){
                                  str_count(ecoreg$ecoreg_code, y)
                                }
                                )

```

Create a column with the number of ecoregions per study

```

ecoreg %<>%
  mutate(
    ecoreg_per_record = rowSums(
      across(
        where(is.numeric)
      )
    )
  )

ecoreg$ecoreg_per_record <- as.factor(ecoreg$ecoreg_per_record)

```

Calculate number of records by number of ecoregions in %

```

ecoreg_per_record_df <- ecoreg %>%
  group_by(ecoreg_per_record) %>%

```

```
summarise(n_records = length(ecoreg_per_record)) %>%
mutate(percentage= round(n_records / sum(n_records) * 100,1))
```

ecoreg_per_record_df

```
## # A tibble: 3 × 3
##   ecoreg_per_record n_records percentage
##   <fct>           <int>       <dbl>
## 1 1             27         90
## 2 2              2         6.7
## 3 3              1         3.3
```

Create a data frame with number of publication per ecoregion

```
ecoreg_tot <-
  ecoreg [, -(length(ecoreg)-1)] %>% # filter out the sum column
  summarise(across(where(is.numeric),
                    list(sum)
                  )
            )
```

```
ecoreg_tot <- reshape2::melt(ecoreg_tot, id.vars = NULL)
```

```
colnames(ecoreg_tot) <- c("Ecoregion", "Count")
```

```
ecoreg_tot %<>%
  mutate_at("Ecoregion",
            stringr::str_replace, "_1", "")
```

Add count column to Meow and set 0 to all the ecoregions where no study was conducted

```
colnames(ecoreg_tot)[colnames(ecoreg_tot) == 'Ecoregion'] <- 'ECO_CODE'
ecoreg_tot$ECO_CODE <- as.character(ecoreg_tot$ECO_CODE)
meow$ECO_CODE <- as.character(meow$ECO_CODE)
```

```
ecoreg_tot <-
  left_join(meow, ecoreg_tot, by = 'ECO_CODE') %>%
  mutate(Count = replace(Count, is.na(Count), 0))
```

Export count per ecoregion to csv for the map in QGIS

```
writexl::write_xlsx(ecoreg_tot,
  here("Extracted_data_analysis", "polar", "Output-Transformed_data", "
eco_reg_tot.xlsx"))
```

Realms

```
realms <- ecoreg_tot %>%
  group_by(RLM_CODE, REALM) %>%
  summarise(Count=sum(Count))
```

```
## `summarise()` has regrouped the output.
## i Summaries were computed grouped by RLM_CODE and REALM.
## i Output is grouped by RLM_CODE.
## i Use `summarise(groups = "drop_last")` to silence this message.
## i Use `summarise(.by = c(RLM_CODE, REALM))` for per-operation grouping
## (`?dplyr::dplyr_by`) instead.

realms %<>%
  rename(Code = RLM_CODE,
         Realm = REALM)

writexl::write_xlsx(realms,
  here("Extracted_data_analysis", "polar", "Output-Transformed_data", "
  Realm_count.xlsx"))
```

Export geographical distribution summary tables

```
writexl::write_xlsx(list(realms, prov_tot, prov_per_record_df, ecoreg_tot, ec
oreg_per_record_df),
  here("Extracted_data_analysis", "polar", "Output-Transformed_data", "
  geographical_distribution_summary.xlsx"))
```

COUNT COLUMNS WITH COMBO

Create a subset of columns

```
count_columns <- polar %>%
  select(File,
         meso_substrate,
         habit,
         environmentalPosition,
         observationalORexperimental,
         taxonomicTargetLevel,
         indicators,
         collectedDataType,
         molecularAnalyses)
```

Prepare data set with columns that need count only and add percentage column

```
count_columns[, ] <- lapply(count_columns[, ], as.factor)

count_results <- list()

for (col_name in colnames(count_columns)) {
  counts <- count(count_columns, !!sym(col_name))

  count_results[[col_name]] <- counts

  colnames(count_results[[col_name]]) <- c(col_name, "Count")
}
```

```

count_results[[col_name]] %<>%
  filter(
    if_all(.cols = everything(), ~ !grepl("notapp", .x))
  ) %>%
  tidyr::drop_na()

count_results[[col_name]] %<>%
  mutate(Percentage= round(Count / sum(Count) * 100,2))
}

rm(col_name, counts)

count_results <- count_results[names(count_results) != "File"]

```

Export summary tables

```

library(openxlsx)

write.xlsx(count_results,
  file = here("Extracted_data_analysis",
    "polar",
    "Output-Transformed_data",
    "summary_counts_question_1-3.xlsx"))

```

Create treemaps charts

```

treemap_func = function(dat,var) {
  ggplot(dat,
    aes(area=Count,
      fill=.data[[var]],
      line = "black",
      label=paste0(.data[[var]],
        " \n ",Percentage,"%"))
  )+
  treemapify::geom_treemap(layout = "squarified")+
  treemapify::geom_treemap_text(place = "centre",
    size = 20)+
  labs(title = var)+
  theme(plot.title = element_text(hjust = 0.5, size= 20),
    legend.position = "none")
}

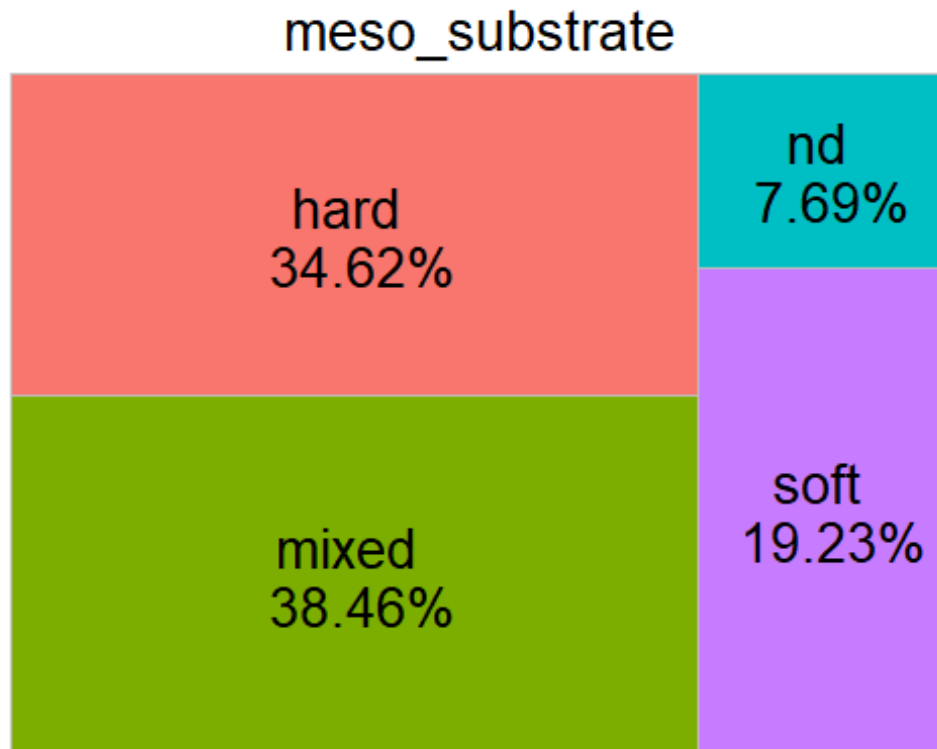
treemap_counts <- list()

for (col_name in colnames(count_columns[,-1])) {
  treemap_counts[[col_name]] <- treemap_func(dat= count_results[[col_name]],
    var = col_name)
}

```

```
}
```

```
treemap_counts[[1]]
```



Save all tree maps plots

```
lapply(names(treemap_counts), function(x){
  ggsave(filename=paste0(x, ".tiff"),
    plot=treemap_counts[[x]],
    units = "in", width=12, height=4, dpi=300, compression = 'lzw',
    path = here("Extracted_data_analysis",
      "polar",
      "Output-Visualization",
      "Treemap_counts"),
    create.dir = T
  )
})
```

```
lapply(names(treemap_counts), function(x){
  ggsave(filename=paste0(x, ".png"),
    plot=treemap_counts[[x]],
    units = "in", width=12, height=4, dpi=300,
    path = here("Extracted_data_analysis",
      "polar",
      "Output-Visualization",
```

```

        "Treemap_counts"),
    create.dir = T
  )
})

```

COUNT METHODS (PURELY TAXONOMIC STUDIES EXCLUDED)

Remove purely taxonomic studies and subset columns (taxa++ was assigned to studies that led to the description of new species but where also ecological e.g. describing community for a given taxonomic group)

```

methods <- polar %>%
  filter(taxonomic_publication == "taxa++" | is.na(taxonomic_publication) )
  %>%
  select(File,
    meso_design,
    meso_replicates,
    temporalReplicates,
    abioticVariablesCollection,
    abioticVariables)

```

```
count_columns_method <- methods
```

Prepare data set with columns that need count only and add percentage column

```
count_columns_method[, ] <- lapply(count_columns_method[, ], as.factor)
```

```
count_results_method <- list()
```

```

for (col_name in colnames(count_columns_method)) {
  counts <- count(count_columns_method, !!sym(col_name))

```

```
count_results_method[[col_name]] <- counts
```

```
colnames(count_results_method[[col_name]]) <- c(col_name, "Count")
```

```
# remove rows containing "notapp"
```

```

count_results_method[[col_name]] %<>%
  filter(
    if_all(.cols = everything(), ~ !grepl("notapp", .x))
  ) %>%
  tidyr::drop_na()

```

```

count_results_method[[col_name]] %<>%
  mutate(Percentage= round(Count / sum(Count) * 100,1))

```

```

}

rm(col_name, counts)

count_results_method <- count_results_method[names(count_results_method) != "
File"]

```

Export summary tables

```

library(openxlsx)

write.xlsx(count_results_method,
           file = here("Extracted_data_analysis", "polar", "Output-Transforme
d_data",
                      "summary_counts_question_4.xlsx"))

```

Create treemaps charts

```

treemap_func = function(dat,var) {
  ggplot(dat,
    aes(area=Count,
        fill=.data[[var]],
        line = "black",
        label=paste0(.data[[var]],
                     " \n ",Percentage,"%"))
  )+
  treemapify::geom_treemap(layout = "squarified")+
  treemapify::geom_treemap_text(place = "centre",
                                size = 20)+
  labs(title = var)+
  theme(plot.title = element_text(hjust = 0.5, size= 20),
        legend.position = "none")
}

treemap_counts <- list()

for (col_name in colnames(count_columns_method[, -1])) {

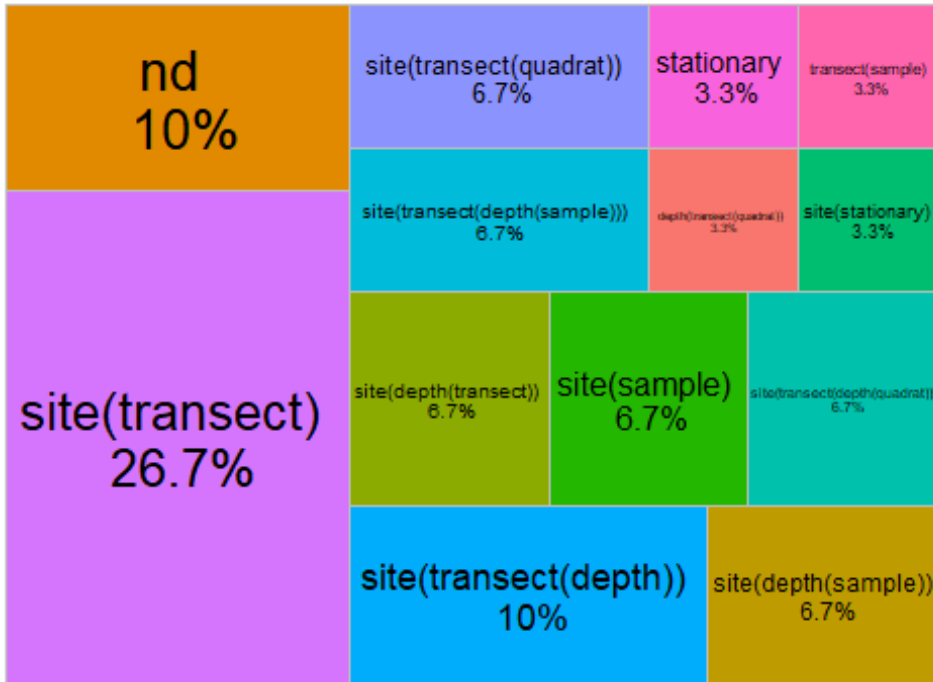
  treemap_counts[[col_name]] <- treemap_func(dat= count_results_method[[col_n
ame]], var = col_name)

}

treemap_counts[[1]]

```

meso_design



Save all tree maps plots

```
lapply(names(treemap_counts), function(x){
  ggsave(filename=paste0("treemap",x,".tiff"),
    plot=treemap_counts[[x]],
    units = "in", width=12, height=4, dpi=300, compression = 'lzw',
    path = here("Extracted_data_analysis","polar",
      "Output-Visualization","Treemaps_methods"),
    create.dir = T
  )
})

lapply(names(treemap_counts), function(x){
  ggsave(filename=paste0("treemap",x,".png"),
    plot=treemap_counts[[x]],
    units = "in", width=12, height=4, dpi=300,
    path = here("Extracted_data_analysis", "polar", "Output-Visualization", "Treemaps_methods"),
    create.dir = T
  )
})
```

COUNT COLUMNS SPLIT

Create a list of data frame splitting values

```
split_columns <- polar %>%
  select(meso_substrate,
         habit,
         environmentalPosition,
         observationalORexperimental,
         taxonomicTargetLevel,
         indicators,
         collectedDataType,
         molecularAnalyses)

split_columns[, ] <- lapply(split_columns[, ], as.factor)
```

Prepare data set with columns

```
split_results <- list()

for (col_name in colnames(split_columns)) {
  # Split the values of each column by "+"
  lvls <- unique(unlist(str_split(split_columns[[col_name]], "\\+")))

  # Initialize a list to store counts for each level
  split_counts <- sapply(lvls, function(y) {
    # Count how many times each unique level appears in the column
    str_count(split_columns[[col_name]], fixed(y))
  })

  split_results[[col_name]] <- split_counts
}

rm(lvls)
```

Add percentage column

```
split_tot <- list()

for(col_name in colnames(split_columns)) {
  split_results[[col_name]] <- as.data.frame(split_results[[col_name]])

  split_results[[col_name]] %<>% # delete rows with missing values
    select(where(function(x) any(!is.na(x)))) %>%
    select(-any_of("notapp")) %>%
    tidyr::drop_na()

  split_tot[[col_name]] <- as.data.frame(split_results[[col_name]]) %>%
    summarise(across(where(is.numeric), list(sum)))

  split_tot[[col_name]] <- reshape2::melt(split_tot[[col_name]], id.vars = N
```

```

ULL)

colnames(split_tot[[col_name]]) <- c(col_name, "Count")

split_tot[[col_name]] %<>%
  mutate_at(col_name,
            stringr::str_replace, "_1", "")

split_tot[[col_name]] %<>%
  mutate(Percentage= round(Count / nrow(split_results[[col_name]] ) * 100,
2))
}

rm(col_name)

```

Export summary tables

```

library(openxlsx)

write.xlsx(split_tot,
           file = here("Extracted_data_analysis", "pol
ar", "Output-Transformed_data",
           "summary_split_counts_question_1_3.xlsx"))

```

Create bar charts for each collected parameter

```

barchart_func = function(dat,var) {

  ggplot(dat,
        aes(x = reorder(.data[[var]], Count), y = Count, fill= .data[[var]]))
+
  geom_bar(stat = "identity", fill = "#0097A7") +
  geom_text(
    data = dat %>% filter(Percentage >= 2),
    aes(label = paste0(round(Percentage, 0), "%")),
    position = position_stack(vjust =0.5),
    color = "black",
    size = 2.5
  )+
  labs(y = "# Publications", x = var) +
  theme_minimal() +
  coord_flip()+ # optional: flips for horizontal bars
  theme(legend.position = "none")

}

barchart_counts <- list()

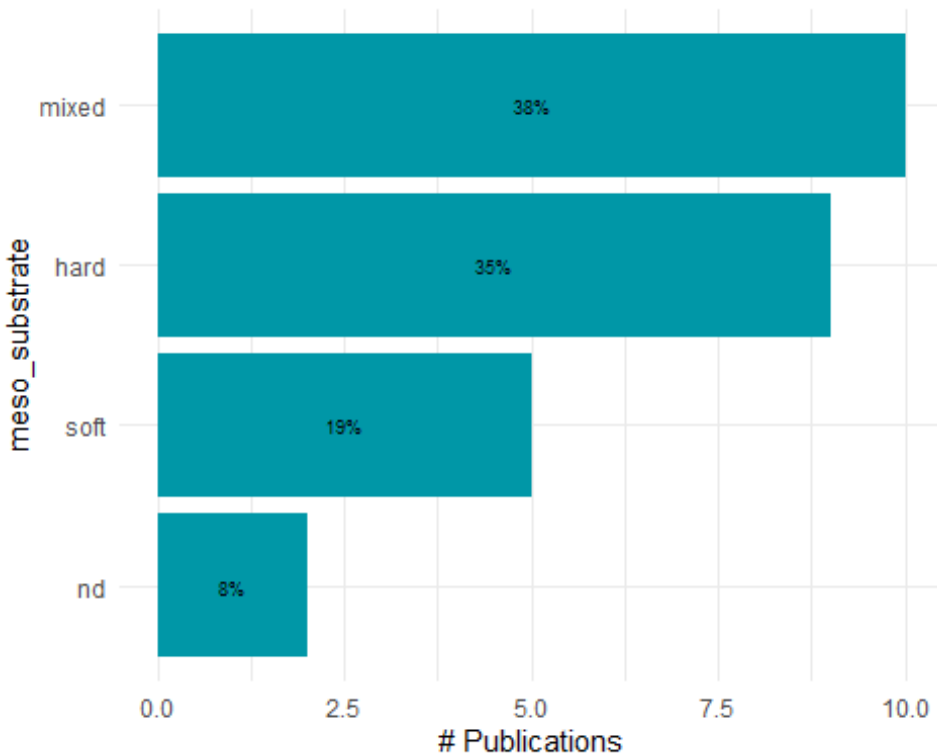
```

```

for (col_name in colnames(split_columns)) {
  barchart_counts[[col_name]] <- barchart_func(dat= split_tot[[col_name]], va
r = as.character(col_name))
}

barchart_counts[[1]]

```



COUNT METHODS (PURELY TAXONOMIC STUDIES EXCLUDED)

Create data frame with subset

```

split_columns_methods <- methods
split_columns_methods[,] <- lapply(split_columns_methods[,], as.factor)

```

Prepare data set with columns

```

split_results_methods <- list()

for (col_name in colnames(split_columns_methods)) {

```

```

# Split the values of each column by "+"
lvls <- unique(unlist(str_split(split_columns_methods[[col_name]], "\\+")))

# Initialize a list to store counts for each level
split_counts_methods <- sapply(lvls, function(y) {
  # Count how many times each unique level appears in the column
  str_count(split_columns_methods[[col_name]], fixed(y))
})

split_results_methods[[col_name]] <- split_counts_methods
}

rm(lvls)

```

Add percentage column

```

split_tot_methods <- list()

for(col_name in colnames(split_columns_methods)) {
  split_results_methods[[col_name]] <- as.data.frame(split_results_methods[[col_name]])

  split_results_methods[[col_name]] %<>% # delete rows with missing values
    select(where(function(x) any(!is.na(x)))) %>%
    select(-any_of("notapp")) %>%
    tidyr::drop_na()

  split_tot_methods[[col_name]] <- as.data.frame(split_results_methods[[col_name]]) %>%
    summarise(across(where(is.numeric), list(sum)))

  split_tot_methods[[col_name]] <- reshape2::melt(split_tot_methods[[col_name]], id.vars = NULL)

  colnames(split_tot_methods[[col_name]]) <- c(col_name, "Count")

  split_tot_methods[[col_name]] %<>%
    mutate_at(col_name,
              stringr::str_replace, "_1", "")

  split_tot_methods[[col_name]] %<>%
    mutate(Percentage= round(Count / nrow(split_results_methods[[col_name]]) * 100,2))
}

```

```
rm(col_name)
```

Export summary tables

```
library(openxlsx)

write.xlsx(split_results_methods,
           file = here("Extracted_data_analysis", "Output-Transformed_data", "pol
ar", "summary_split_counts_question_4.xlsx"))
```

Create bar charts per each parameter collected

```
barchart_func = function(dat,var) {

  ggplot(dat,
         aes(x = reorder(.data[[var]], Count), y = Count, fill= .data[[var]]))
+
  geom_bar(stat = "identity", fill = "#0097A7") +
  geom_text(
    data = dat %>% filter(Percentage >= 2),
    aes(label = paste0(round(Percentage, 0), "%"),
        position = position_stack(vjust =0.5),
        color = "black",
        size = 2.5
    )+
    labs(y = "# Publications", x = var) +
    theme_minimal() +
    coord_flip()+ # optional: flips for horizontal bars
    theme(legend.position = "none")
  )
}

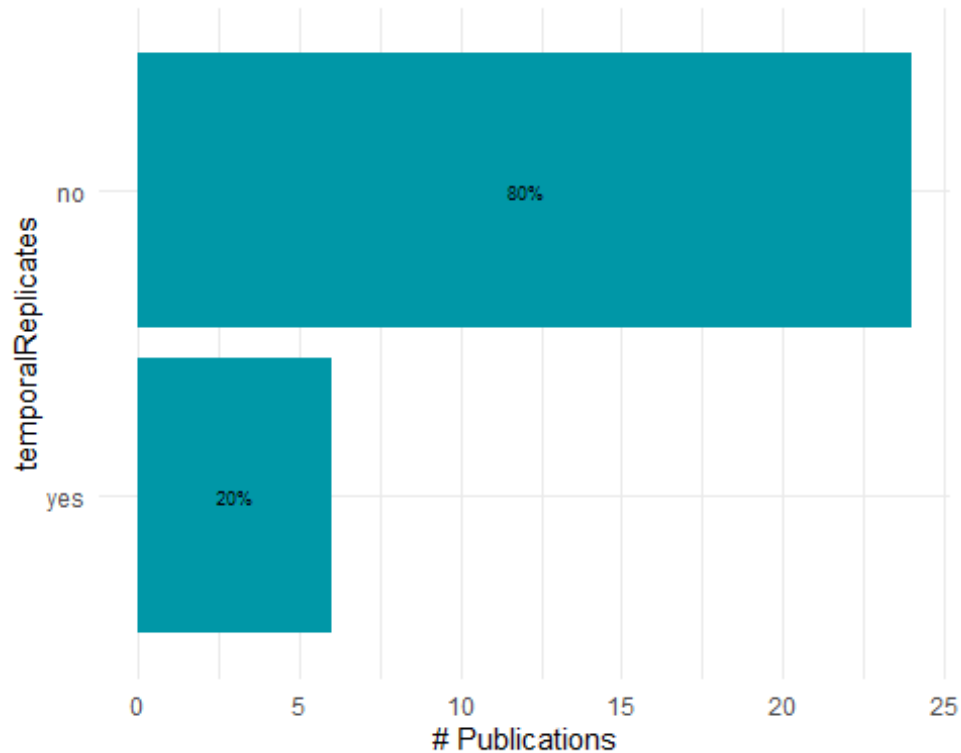
barchart_split_methods <- list()

for (col_name in colnames(split_columns_methods)) {

  barchart_split_methods[[col_name]] <- barchart_func(dat= split_tot_methods
[[col_name]], var = as.character(col_name))

}

barchart_split_methods[[4]]
```



Export bar charts in a dedicated folder

```
lapply(names(barchart_counts), function(x){
  ggsave(filename=paste0("bar_",x,".tiff"),
    plot=barchart_counts[[x]],
    units = "in", width=12, height=4, dpi=300, compression = 'lzw',
    path = here("Extracted_data_analysis", "polar",
      "Output-Visualization", "Barcharts_split_methods"),
    create.dir = T
  )
})

lapply(names(barchart_counts), function(x){
  ggsave(filename=paste0("bar_",x,".png"),
    plot=barchart_counts[[x]],
    units = "in", width=12, height=4, dpi=300,
    path = here("Extracted_data_analysis", "polar",
      "Output-Visualization", "Barcharts_split"), create.d
    ir = T
  )
})
```

PUBLICATIONS PER DEPTH

```
polar$minimumSurveyedDepthInMeters <- as.numeric(polar$minimumSurveyedDepthInMeters)
```

```

## Warning: NAs introduced by coercion

polar$maximumSurveyedDepthInMeters <- as.numeric(polar$maximumSurveyedDepthInMeters)

## Warning: NAs introduced by coercion

# Define depth bins
bins <- seq(31, 151, by = 10)

# Create a data frame to hold bin counts
pubs_depth <- data.frame(
  bin_start = bins[-length(bins)],
  bin_end = bins[-1]-1,
  count = 0
)

# For each bin, count how many studies cover that depth range
for (i in 1:nrow(pubs_depth)) {
  bin_start <- pubs_depth$bin_start[i]
  bin_end <- pubs_depth$bin_end[i]

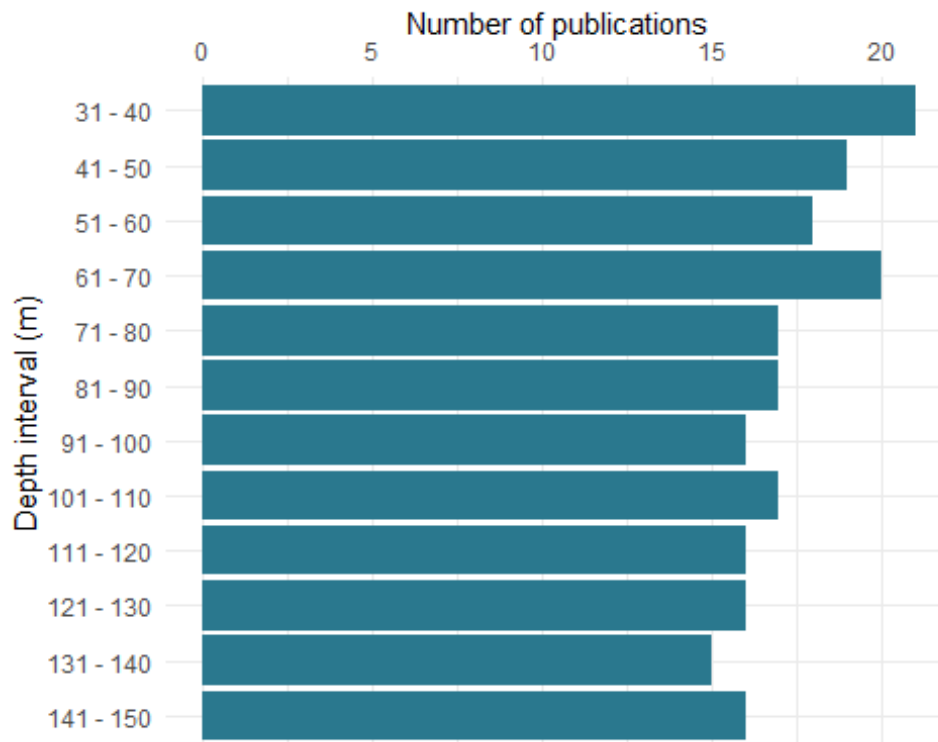
  pubs_depth$count[i] <- sum(polar$maximumSurveyedDepthInMeters >= bin_start
& polar$minimumSurveyedDepthInMeters < bin_end, na.rm = TRUE)
}

pubs_depth$depth_bin <- paste(pubs_depth$bin_start, "-", pubs_depth$bin_end)

pubs_depth$depth_bin <- reorder(pubs_depth$depth_bin, -pubs_depth$bin_start)

ggplot(pubs_depth, aes(x = factor(depth_bin), y = count)) +
  geom_bar(stat = "identity", fill = "#2a788e") +
  labs(x = "Depth interval (m)", y = "Number of publications") +
  theme_minimal()+
  coord_flip()+
  scale_y_continuous(position="right")

```



Export pubs per depth bar plot

PLATFORM BY DEPTH

Create a platform by depth data frame and keep meso_samplingPlatform

```
platform_depth <- polar %>%
  select(File,
    mininumSurveyedDepthInMeters,
    maximumSurveyedDepthInMeters,
    meso_direct_indirect,
    meso_samplingPlatform,
    max_depth_hov, min_depth_hov,
    max_depth_rov, min_depth_rov,
    max_depth_ccr, min_depth_ccr,
    max_depth_open, min_depth_open,
    max_depth_scuba, min_depth_scuba
  )
```

Split the platform column in one binary column per platform

```
platform_lvls <-
  unique(
    unlist(
      str_split(platform_depth$meso_samplingPlatform, "\\+")
    )
  )
```

```

    )

platform_lvls<- platform_lvls[! platform_lvls %in% c("",NA,"notapp","", "nd")]

platform_depth[,platform_lvls] <- sapply(platform_lvls,
                                          function(y){
                                            str_count(platform_depth$meso_samplin
gPlatform, y)
                                          })

rm(platform_lvls)

platform_depth[, -c(1:15)] <- lapply(platform_depth[, -c(1:15)], as.numeric)

platform_depth[, c(1:15)] <- lapply(platform_depth[, c(1:15)], as.character)

platform_depth %<>%
  rename(open_direct = open) %>%
  select(-meso_samplingPlatform)

```

Create a summary table with # studies per platform

```

platform_summary <- platform_depth %>%
  summarise(across(where(is.numeric), ~ sum(.x, na.rm = TRUE)))

platform_summary <- as.data.frame(t(platform_summary))

platform_summary %<>%
  mutate(Platform = rownames(platform_summary)) %>%
  rename(Count = V1) %>%
  relocate(Count, .after = where(is.character))

platform_summary

##           Platform Count
## rov           rov     17
## hov           hov      6
## dredge         dredge   4
## acou           acou     5
## drop           drop     8
## fishery        fishery   2
## grab           grab      3
## trawl          trawl     1
## tcs            tcs       2
## open_direct    open_direct 1
## station        station   1
## bruv           bruv      1

```

Create a column with the number of platforms per study

```
platform_per_record <- platform_depth %>%
  filter(meso_direct_indirect != "nd") %>%
  mutate(
    platform_per_record = rowSums(
      across(
        where(is.numeric)
      )
    )
  )

platform_per_record$platform_per_record <- as.factor(platform_per_record$platform_per_record)
```

Calculate number of records by number of platforms in %

```
platform_per_record <- platform_per_record %>%
  group_by(platform_per_record) %>%
  summarise(n_records = length(platform_per_record)) %>%
  mutate(percentage = round(n_records / sum(n_records) * 100, 1))
```

platform_per_record

```
## # A tibble: 4 × 3
##   platform_per_record n_records percentage
##   <fct>              <int>         <dbl>
## 1 1                  17          56.7
## 2 2                   6           20
## 3 3                   6           20
## 4 4                   1           3.3
```

Keep most common visual platforms only

```
platform_depth_mod <- cbind(platform_depth[,c(1:14)],
                             platform_depth$open_direct,
                             platform_depth$hov,
                             platform_depth$rov)

names(platform_depth_mod) <- gsub("platform_depth\\$", "", names(platform_depth_mod))
```

Set max depth of studies with max depth > 150 to 150 so they fit into the plot

```
platform_depth_long <- within(platform_depth_mod, rm(mininumSurveyedDepthInMeters, maximumSurveyedDepthInMeters, max_depth_direct, meso_direct_indirect, hov, rov, open_direct, ccr_direct, scuba_direct))

platform_depth_long[, -1] <- lapply(platform_depth_long[, -1], as.numeric)

str(platform_depth_long)
```

```
## 'data.frame':    30 obs. of  11 variables:
## $ File           : chr  "LaudienOrchard_2012_significance_depth_substratu
m" "HamadaEtAl_1986_Observation_marine_benthic.md" "AnsellEtAl_1998_Swimming_
Antarctic_scallop.md" "ZhouShirley_1998_submersible_study_red.md" ...
## $ max_depth_hov   : num  NA NA NA 200 NA NA NA 99 NA 50 ...
## $ min_depth_hov   : num  NA NA NA 10 NA NA NA 30 NA 38 ...
## $ max_depth_rov    : num  200 200 80 NA 212 ...
## $ min_depth_rov    : num  30 16 30 NA 28 0 108 NA 143 NA ...
## $ max_depth_ccr    : num  NA NA NA NA NA NA NA NA NA NA ...
## $ min_depth_ccr    : num  NA NA NA NA NA NA NA NA NA NA ...
## $ max_depth_open   : num  NA NA NA NA NA NA NA NA NA NA ...
## $ min_depth_open   : num  NA NA NA NA NA NA NA NA NA NA ...
## $ max_depth_scuba  : num  NA NA NA NA NA NA NA NA NA NA ...
## $ min_depth_scuba  : num  NA NA NA NA NA NA NA NA NA NA ...

platform_depth_long %<>%
  mutate(max_depth_scuba = case_when(max_depth_scuba >= 150 ~ 150,
                                     TRUE ~ max_depth_scuba)
  ) %>%
  mutate(max_depth_hov = case_when(max_depth_hov >= 150 ~ 150,
                                   TRUE ~ max_depth_hov)
  ) %>%
  mutate(max_depth_rov = case_when(max_depth_rov >= 150 ~ 150,
                                   TRUE ~ max_depth_rov)
  ) %>%
  mutate(max_depth_ccr = case_when(max_depth_ccr >= 150 ~ 150,
                                   TRUE ~ max_depth_ccr)) %>%
  mutate(max_depth_open = case_when(max_depth_open >= 150 ~ 150,
                                    TRUE ~ max_depth_open))
```

Set min depth of studies with min depth <30 to 30 so they fit into the plot

```
platform_depth_long %<>%
  mutate(min_depth_scuba = case_when(min_depth_scuba <= 30 ~ 30,
                                     TRUE ~ min_depth_scuba)
  ) %>%
  mutate(min_depth_hov = case_when(min_depth_hov <= 30 ~ 30,
                                   TRUE ~ min_depth_hov)
  ) %>%
  mutate(min_depth_rov = case_when(min_depth_rov <= 30 ~ 30,
                                   TRUE ~ min_depth_rov)
  ) %>%
  mutate(min_depth_ccr = case_when(min_depth_ccr <= 30 ~ 30,
                                   TRUE ~ min_depth_ccr)) %>%
  mutate(min_depth_open = case_when(min_depth_open <= 30 ~ 30,
                                    TRUE ~ min_depth_open))
```

Convert platform_depth_long to long format

```
platform_min_depth_long <- platform_depth_long %>%
  select(-c(max_depth_hov, max_depth_rov, max_depth_open, max_depth_scuba, ma
```

```

x_depth_ccr))

platform_min_depth_long[,c(1,2)] <- lapply(platform_min_depth_long[,c(1,2)],
as.factor)

platform_min_depth_long %<>%
  reshape2::melt(platform_min_depth_long,
    id.vars = c("File"),
    measure.vars = grep("^min_depth_|^max_depth_",
      names(platform_min_depth_long),
      value = TRUE),
    variable.name = "Platform",
    value.name = "min_depth"
  )

## Warning: attributes are not identical across measure variables; they will
## be
## dropped

platform_max_depth_long <- platform_depth_long %>%
  select(-c(min_depth_hov, min_depth_rov, min_depth_open, min_depth_scuba,
min_depth_ccr))

platform_max_depth_long %<>%
  reshape2::melt(platform_max_depth_long,
    id.vars = c("File"),
    measure.vars = grep("^min_depth_|^max_depth_",
      names(platform_max_depth_long),
      value = TRUE),
    variable.name = "Platform",
    value.name = "max_depth"
  )

platform_depth_lat_long <- cbind(platform_min_depth_long, platform_max_depth_
long$max_depth)

rm(platform_min_depth_long, platform_max_depth_long)

platform_depth_lat_long

##
##
rm
## 1      LaudienOrchard_2012_significance_depth_substratum  min_depth_h
ov
## 2      HamadaEtAl_1986_Observation_marine_benthic.md    min_depth_h
ov
## 3      AnsellEtAl_1998_Swimming_Antarctic_scallop.md    min_depth_h
ov
## 4      ZhouShirley_1998_submersible_study_red.md        min_depth_h

```

OV		
## 5	AmbroseEtAl_2001_Role_echinoderms_benthic.md	min_depth_h
OV		
## 6	AmbroseJr.EtAl_2005_subice_algal_community.md	min_depth_h
OV		
## 7	TeixidoEtAl_2006_Observations_asexual_reproductive.md	min_depth_h
OV		
## 8	TeichertEtAl_2012_Rhodolith_beds_Corallinalesa.md	min_depth_h
OV		
## 9	GuttEtAl_2013_Shifts_Antarctic_megabenthic.md	min_depth_h
OV		
## 10	TeichertEtAl_2014_Arctic_rhodolith_beds.md	min_depth_h
OV		
## 11	TeichertFreiwald_2014_Polar_coralline_algal.md	min_depth_h
OV		
## 12	CrookDavoren_2016_Influence_spawning_capelin.md	min_depth_h
OV		
## 13	YessonEtAl_2017_impact_trawling_epibenthic.md	min_depth_h
OV		
## 14	WatsonEtAl_2018_Patterns_distribution_abundance.md	min_depth_h
OV		
## 15	CooperEtAl_2019_video_seafloor_survey.md	min_depth_h
OV		
## 16	DinnEtAl_2019_Sponge_Porifera_fauna.md	min_depth_h
OV		
## 17	PierrejeanEtAl_2020_Influence_deepwater_corals.md	min_depth_h
OV		
## 18	WisshakNeumann_2020_Dead_urchin_walking.md	min_depth_h
OV		
## 19	CastellanEtAl_2021_Visual_imaging_benthic.md	min_depth_h
OV		
## 20	KimEtAl_2021_Patterns_drivers_implications.md	min_depth_h
OV		
## 21	KrawczykEtAl_2021_First_highresolution_benthic.md	min_depth_h
OV		
## 22	TeichertEtAl_2021_Microplastic_contamination_drilling.md	min_depth_h
OV		
## 23	KimCazenave_2022_Benthic_community_descriptions.md	min_depth_h
OV		
## 24	MedelyteEtAl_2022_Application_underwater_imagery.md	min_depth_h
OV		
## 25	RangeleyEtAl_2022_Megabenthic_biodiversity_culturally.md	min_depth_h
OV		
## 26	RobinsonEtAl_2022_New_confirmed_depth.md	min_depth_h
OV		
## 27	SchimaniEtAl_2022_Video_survey_deep.md	min_depth_h
OV		
## 28	WisshakEtAl_2022_Ten_Years_longterm.md	min_depth_h
OV		
## 29	DejaEtAl_2023_New_strategies_new.md	min_depth_h

OV		
## 30	SiglerEtAl_2023_Alaska_deepsea_coral.md	min_depth_h
OV		
## 31	LaudienOrchard_2012_significance_depth_substratum	min_depth_r
OV		
## 32	HamadaEtAl_1986_Observation_marine_benthic.md	min_depth_r
OV		
## 33	AnsellEtAl_1998_Swimming_Antarctic_scallop.md	min_depth_r
OV		
## 34	ZhouShirley_1998_submersible_study_red.md	min_depth_r
OV		
## 35	AmbroseEtAl_2001_Role_echinoderms_benthic.md	min_depth_r
OV		
## 36	AmbroseJr.EtAl_2005_subice_algal_community.md	min_depth_r
OV		
## 37	TeixidoEtAl_2006_Observations_asexual_reproductive.md	min_depth_r
OV		
## 38	TeichertEtAl_2012_Rhodolith_beds_Corallinalesa.md	min_depth_r
OV		
## 39	GuttEtAl_2013_Shifts_Antarctic_megabenthic.md	min_depth_r
OV		
## 40	TeichertEtAl_2014_Arctic_rhodolith_beds.md	min_depth_r
OV		
## 41	TeichertFreiwald_2014_Polar_coralline_algal.md	min_depth_r
OV		
## 42	CrookDavoren_2016_Influence_spawning_capelin.md	min_depth_r
OV		
## 43	YessonEtAl_2017_impact_trawling_epibenthic.md	min_depth_r
OV		
## 44	WatsonEtAl_2018_Patterns_distribution_abundance.md	min_depth_r
OV		
## 45	CooperEtAl_2019_video_seafloor_survey.md	min_depth_r
OV		
## 46	DinnEtAl_2019_Sponge_Porifera_fauna.md	min_depth_r
OV		
## 47	PierrejeanEtAl_2020_Influence_deepwater_corals.md	min_depth_r
OV		
## 48	WisshakNeumann_2020_Dead_urchin_walking.md	min_depth_r
OV		
## 49	CastellanEtAl_2021_Visual_imaging_benthic.md	min_depth_r
OV		
## 50	KimEtAl_2021_Patterns_drivers_implications.md	min_depth_r
OV		
## 51	KrawczykEtAl_2021_First_highresolution_benthic.md	min_depth_r
OV		
## 52	TeichertEtAl_2021_Microplastic_contamination_drilling.md	min_depth_r
OV		
## 53	KimCazenave_2022_Benthic_community_descriptions.md	min_depth_r
OV		
## 54	MedelyteEtAl_2022_Application_underwater_imagery.md	min_depth_r

ov		
## 55	RangeleyEtAl_2022_Megabenthic_biodiversity_culturally.md	min_depth_r
ov		
## 56	RobinsonEtAl_2022_New_confirmed_depth.md	min_depth_r
ov		
## 57	SchimaniEtAl_2022_Video_survey_deep.md	min_depth_r
ov		
## 58	WisshakEtAl_2022_Ten_Years_longterm.md	min_depth_r
ov		
## 59	DejaEtAl_2023_New_strategies_new.md	min_depth_r
ov		
## 60	SiglerEtAl_2023_Alaska_deepsea_coral.md	min_depth_r
ov		
## 61	LaudienOrchard_2012_significance_depth_substratum	min_depth_c
cr		
## 62	HamadaEtAl_1986_Observation_marine_benthic.md	min_depth_c
cr		
## 63	AnsellEtAl_1998_Swimming_Antarctic_scallop.md	min_depth_c
cr		
## 64	ZhouShirley_1998_submersible_study_red.md	min_depth_c
cr		
## 65	AmbroseEtAl_2001_Role_echinoderms_benthic.md	min_depth_c
cr		
## 66	AmbroseJr.EtAl_2005_subice_algal_community.md	min_depth_c
cr		
## 67	TeixidoEtAl_2006_Observations_asexual_reproductive.md	min_depth_c
cr		
## 68	TeichertEtAl_2012_Rhodolith_beds_Corallinalesa.md	min_depth_c
cr		
## 69	GuttEtAl_2013_Shifts_Antarctic_megabenthic.md	min_depth_c
cr		
## 70	TeichertEtAl_2014_Arctic_rhodolith_beds.md	min_depth_c
cr		
## 71	TeichertFreiwald_2014_Polar_coralline_algal.md	min_depth_c
cr		
## 72	CrookDavoren_2016_Influence_spawning_capelin.md	min_depth_c
cr		
## 73	YessonEtAl_2017_impact_trawling_epibenthic.md	min_depth_c
cr		
## 74	WatsonEtAl_2018_Patterns_distribution_abundance.md	min_depth_c
cr		
## 75	CooperEtAl_2019_video_seafloor_survey.md	min_depth_c
cr		
## 76	DinnEtAl_2019_Sponge_Porifera_fauna.md	min_depth_c
cr		
## 77	PierrejeanEtAl_2020_Influence_deepwater_corals.md	min_depth_c
cr		
## 78	WisshakNeumann_2020_Dead_urchin_walking.md	min_depth_c
cr		
## 79	CastellanEtAl_2021_Visual_imaging_benthic.md	min_depth_c

cr		
## 80	KimEtAl_2021_Patterns_drivers_implications.md	min_depth_c
cr		
## 81	KrawczykEtAl_2021_First_highresolution_benthic.md	min_depth_c
cr		
## 82	TeichertEtAl_2021_Microplastic_contamination_drilling.md	min_depth_c
cr		
## 83	KimCazenave_2022_Benthic_community_descriptions.md	min_depth_c
cr		
## 84	MedelyteEtAl_2022_Application_underwater_imagery.md	min_depth_c
cr		
## 85	RangeleyEtAl_2022_Megabenthic_biodiversity_culturally.md	min_depth_c
cr		
## 86	RobinsonEtAl_2022_New_confirmed_depth.md	min_depth_c
cr		
## 87	SchimaniEtAl_2022_Video_survey_deep.md	min_depth_c
cr		
## 88	WisshakEtAl_2022_Ten_Years_longterm.md	min_depth_c
cr		
## 89	DejaEtAl_2023_New_strategies_new.md	min_depth_c
cr		
## 90	SiglerEtAl_2023_Alaska_deepsea_coral.md	min_depth_c
cr		
## 91	LaudienOrchard_2012_significance_depth_substratum	min_depth_op
en		
## 92	HamadaEtAl_1986_Observation_marine_benthic.md	min_depth_op
en		
## 93	AnsellEtAl_1998_Swimming_Antarctic_scallop.md	min_depth_op
en		
## 94	ZhouShirley_1998_submersible_study_red.md	min_depth_op
en		
## 95	AmbroseEtAl_2001_Role_echinoderms_benthic.md	min_depth_op
en		
## 96	AmbroseJr.EtAl_2005_subice_algal_community.md	min_depth_op
en		
## 97	TeixidoEtAl_2006_Observations_asexual_reproductive.md	min_depth_op
en		
## 98	TeichertEtAl_2012_Rhodolith_beds_Corallinalesa.md	min_depth_op
en		
## 99	GuttEtAl_2013_Shifts_Antarctic_megabenthic.md	min_depth_op
en		
## 100	TeichertEtAl_2014_Arctic_rhodolith_beds.md	min_depth_op
en		
## 101	TeichertFreiwald_2014_Polar_coralline_algal.md	min_depth_op
en		
## 102	CrookDavoren_2016_Influence_spawning_capelin.md	min_depth_op
en		
## 103	YessonEtAl_2017_impact_trawling_epibenthic.md	min_depth_op
en		
## 104	WatsonEtAl_2018_Patterns_distribution_abundance.md	min_depth_op

en		
## 105	CooperEtAl_2019_video_seafloor_survey.md	min_depth_op
en		
## 106	DinnEtAl_2019_Sponge_Porifera_fauna.md	min_depth_op
en		
## 107	PierrejeanEtAl_2020_Influence_deepwater_corals.md	min_depth_op
en		
## 108	WisshakNeumann_2020_Dead_urchin_walking.md	min_depth_op
en		
## 109	CastellanEtAl_2021_Visual_imaging_benthic.md	min_depth_op
en		
## 110	KimEtAl_2021_Patterns_drivers_implications.md	min_depth_op
en		
## 111	KrawczykEtAl_2021_First_highresolution_benthic.md	min_depth_op
en		
## 112	TeichertEtAl_2021_Microplastic_contamination_drilling.md	min_depth_op
en		
## 113	KimCazenave_2022_Benthic_community_descriptions.md	min_depth_op
en		
## 114	MedelyteEtAl_2022_Application_underwater_imagery.md	min_depth_op
en		
## 115	RangeleyEtAl_2022_Megabenthic_biodiversity_culturally.md	min_depth_op
en		
## 116	RobinsonEtAl_2022_New_confirmed_depth.md	min_depth_op
en		
## 117	SchimaniEtAl_2022_Video_survey_deep.md	min_depth_op
en		
## 118	WisshakEtAl_2022_Ten_Years_longterm.md	min_depth_op
en		
## 119	DejaEtAl_2023_New_strategies_new.md	min_depth_op
en		
## 120	SiglerEtAl_2023_Alaska_deepsea_coral.md	min_depth_op
en		
## 121	LaudienOrchard_2012_significance_depth_substratum	min_depth_scu
ba		
## 122	HamadaEtAl_1986_Observation_marine_benthic.md	min_depth_scu
ba		
## 123	AnsellEtAl_1998_Swimming_Antarctic_scallop.md	min_depth_scu
ba		
## 124	ZhouShirley_1998_submersible_study_red.md	min_depth_scu
ba		
## 125	AmbroseEtAl_2001_Role_echinoderms_benthic.md	min_depth_scu
ba		
## 126	AmbroseJr.EtAl_2005_subice_algal_community.md	min_depth_scu
ba		
## 127	TeixidoEtAl_2006_Observations_asexual_reproductive.md	min_depth_scu
ba		
## 128	TeichertEtAl_2012_Rhodolith_beds_Corallinalesa.md	min_depth_scu
ba		
## 129	GuttEtAl_2013_Shifts_Antarctic_megabenthic.md	min_depth_scu

```

ba
## 130      TeichertEtAl_2014_Arctic_rhodolith_beds.md min_depth_scu
ba
## 131      TeichertFreiwald_2014_Polar_coralline_algal.md min_depth_scu
ba
## 132      CrookDavoren_2016_Influence_spawning_capelin.md min_depth_scu
ba
## 133      YessonEtAl_2017_impact_trawling_epibenthic.md min_depth_scu
ba
## 134      WatsonEtAl_2018_Patterns_distribution_abundance.md min_depth_scu
ba
## 135      CooperEtAl_2019_video_seafloor_survey.md min_depth_scu
ba
## 136      DinnEtAl_2019_Sponge_Porifera_fauna.md min_depth_scu
ba
## 137      PierrejeanEtAl_2020_Influence_deepwater_corals.md min_depth_scu
ba
## 138      WisshakNeumann_2020_Dead_urchin_walking.md min_depth_scu
ba
## 139      CastellonEtAl_2021_Visual_imaging_benthic.md min_depth_scu
ba
## 140      KimEtAl_2021_Patterns_drivers_implications.md min_depth_scu
ba
## 141      KrawczykEtAl_2021_First_highresolution_benthic.md min_depth_scu
ba
## 142 TeichertEtAl_2021_Microplastic_contamination_drilling.md min_depth_scu
ba
## 143      KimCazenave_2022_Benthic_community_descriptions.md min_depth_scu
ba
## 144      MedelyteEtAl_2022_Application_underwater_imagery.md min_depth_scu
ba
## 145 RangeleyEtAl_2022_Megabenthic_biodiversity_culturally.md min_depth_scu
ba
## 146      RobinsonEtAl_2022_New_confirmed_depth.md min_depth_scu
ba
## 147      SchimaniEtAl_2022_Video_survey_deep.md min_depth_scu
ba
## 148      WisshakEtAl_2022_Ten_Years_longterm.md min_depth_scu
ba
## 149      DejaEtAl_2023_New_strategies_new.md min_depth_scu
ba
## 150      SiglerEtAl_2023_Alaska_deepsea_coral.md min_depth_scu
ba
##      min_depth platform_max_depth_long$max_depth
## 1      <NA> NA
## 2      <NA> NA
## 3      <NA> NA
## 4      30 150
## 5      <NA> NA
## 6      <NA> NA

```

## 7	<NA>	NA
## 8	30	99
## 9	<NA>	NA
## 10	38	50
## 11	38	50
## 12	<NA>	NA
## 13	<NA>	NA
## 14	<NA>	NA
## 15	<NA>	NA
## 16	<NA>	NA
## 17	<NA>	NA
## 18	<NA>	NA
## 19	<NA>	NA
## 20	<NA>	NA
## 21	<NA>	NA
## 22	30	40
## 23	<NA>	NA
## 24	<NA>	NA
## 25	<NA>	NA
## 26	<NA>	NA
## 27	<NA>	NA
## 28	30	127
## 29	<NA>	NA
## 30	<NA>	NA
## 31	30	150
## 32	30	150
## 33	30	80
## 34	<NA>	NA
## 35	30	150
## 36	30	150
## 37	108	150
## 38	<NA>	NA
## 39	143	150
## 40	<NA>	NA
## 41	<NA>	NA
## 42	30	40
## 43	<NA>	NA
## 44	32	150
## 45	<NA>	NA
## 46	65	65
## 47	127	150
## 48	<NA>	NA
## 49	30	150
## 50	30	90
## 51	<NA>	NA
## 52	<NA>	NA
## 53	40	70
## 54	30	65
## 55	<NA>	NA
## 56	89	102

## 57	30	128
## 58	<NA>	NA
## 59	<NA>	NA
## 60	<NA>	NA
## 61	<NA>	NA
## 62	<NA>	NA
## 63	<NA>	NA
## 64	<NA>	NA
## 65	<NA>	NA
## 66	<NA>	NA
## 67	<NA>	NA
## 68	<NA>	NA
## 69	<NA>	NA
## 70	<NA>	NA
## 71	<NA>	NA
## 72	<NA>	NA
## 73	<NA>	NA
## 74	<NA>	NA
## 75	<NA>	NA
## 76	<NA>	NA
## 77	<NA>	NA
## 78	<NA>	NA
## 79	<NA>	NA
## 80	<NA>	NA
## 81	<NA>	NA
## 82	<NA>	NA
## 83	<NA>	NA
## 84	<NA>	NA
## 85	<NA>	NA
## 86	<NA>	NA
## 87	<NA>	NA
## 88	<NA>	NA
## 89	<NA>	NA
## 90	<NA>	NA
## 91	<NA>	NA
## 92	<NA>	NA
## 93	<NA>	NA
## 94	<NA>	NA
## 95	<NA>	NA
## 96	<NA>	NA
## 97	<NA>	NA
## 98	<NA>	NA
## 99	<NA>	NA
## 100	<NA>	NA
## 101	<NA>	NA
## 102	<NA>	NA
## 103	<NA>	NA
## 104	<NA>	NA
## 105	<NA>	NA
## 106	<NA>	NA

## 107	<NA>	NA
## 108	<NA>	NA
## 109	<NA>	NA
## 110	<NA>	NA
## 111	<NA>	NA
## 112	<NA>	NA
## 113	35	40
## 114	<NA>	NA
## 115	<NA>	NA
## 116	<NA>	NA
## 117	<NA>	NA
## 118	<NA>	NA
## 119	<NA>	NA
## 120	<NA>	NA
## 121	<NA>	NA
## 122	<NA>	NA
## 123	<NA>	NA
## 124	<NA>	NA
## 125	<NA>	NA
## 126	<NA>	NA
## 127	<NA>	NA
## 128	<NA>	NA
## 129	<NA>	NA
## 130	<NA>	NA
## 131	<NA>	NA
## 132	<NA>	NA
## 133	<NA>	NA
## 134	<NA>	NA
## 135	<NA>	NA
## 136	<NA>	NA
## 137	<NA>	NA
## 138	<NA>	NA
## 139	<NA>	NA
## 140	<NA>	NA
## 141	<NA>	NA
## 142	<NA>	NA
## 143	<NA>	NA
## 144	<NA>	NA
## 145	<NA>	NA
## 146	<NA>	NA
## 147	<NA>	NA
## 148	<NA>	NA
## 149	<NA>	NA
## 150	<NA>	NA

```
platform_depth_lat_long %<>%
```

```
  rename(max_depth = `platform_max_depth_long$max_depth`)
```

```
platform_depth_lat_long$Platform <- gsub("min_depth_", "", platform_depth_lat_long$Platform)
```

```
platform_depth_lat_long$Platform <- gsub("max_depth_", "", platform_depth_lat_long$Platform)
```

Set proper labelling for platform

```
platform_depth_lat_long %<>%
  mutate(Platform = case_when(Platform == "open" ~ "OC",
                              Platform == "rov" ~ "ROV",
                              Platform == "hov" ~ "HOV",
                              TRUE ~ Platform)
  ) %>%
  mutate(Platform = as.factor(Platform))
```

Add visual column to reorder platform

```
platform_depth_lat_long %<>%
  mutate(Visual = as.factor(case_when(Platform == "OC" |
                                      Platform == "SCUBA" |
                                      Platform == "CCR" ~ "direct",
                                      TRUE ~ "indirect" )
  )
)
```

```
platform_depth_lat_long %<>%
  mutate(Platform = forcats::fct_reorder(Platform, as.integer(Visual)))
```

Create a bar plot with depth bins by platform and latitudinalRegion

```
depth_bins <- tibble(
  bin_start = seq(-30, -140, by = -10),
  bin_end   = seq(-40, -150, by = -10)
) %>%
  mutate(bin_label = paste0(abs(bin_start), "-", abs(bin_end)))
```

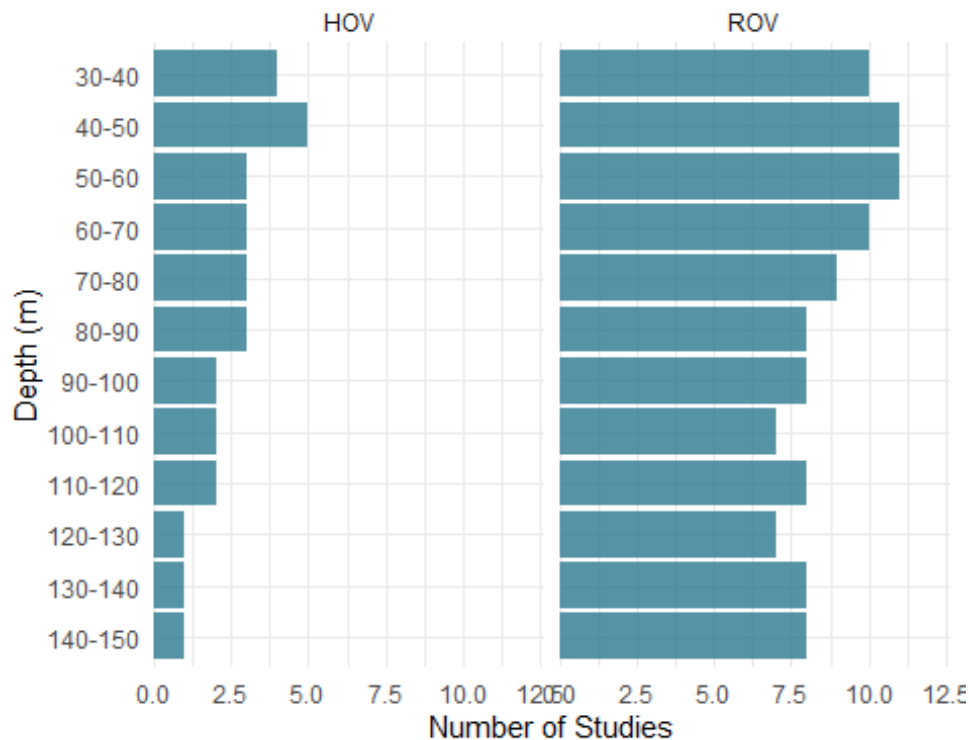
```
platform_depth_lat_long <- platform_depth_lat_long %>%
  mutate(
    min_depth = as.numeric(min_depth), # positive e.g., 30
    max_depth = as.numeric(max_depth)  # positive e.g., 150
  )
```

```
expanded_data <- platform_depth_lat_long %>%
  tidyr::crossing(depth_bins) %>%
  # Convert study range to negative depths: [-max_depth, -min_depth]
  filter(bin_end >= -max_depth, bin_start <= -min_depth) %>%
  mutate(
    bin_label = factor(bin_label, levels = rev(depth_bins$bin_label))
  )
```

Plot with count labels on the right

```
platform_depth_plot <- ggplot(expanded_data, aes(x = bin_label)) +
  geom_bar(position = "stack", alpha= 0.8, fill= "#2a788e") +
  scale_y_continuous(name = "Number of Studies",
                     expand = expansion(mult = c(0, 0.15))) + # Make room for
# text
  scale_x_discrete(name = "Depth (m)") +
  coord_flip() + # Depth on y-axis (shallow at top)
  facet_wrap(~ Platform, nrow = 1) +
  theme_minimal() +
  theme(legend.position = "none")

platform_depth_plot
```



Export bar plot of most common platform per depth interval

```
ggsave("platform_depth_plot.tiff",
       plot = platform_depth_plot,
       units = "in", width = 8, height = 6, dpi = 300, compression = 'lzw',
       path = here ("Extracted_data_analysis", "polar",
                    "Output-Visualization"))

ggsave("platform_depth_plot.png",
       plot = platform_depth_plot,
       units = "in", width = 8, height = 5,
       path = here ("Extracted_data_analysis", "polar",
                    "Output-Visualization"))
```

DIRECT - INDIRECT BY DECADE

```
dir_ind_by_decade <- polar

dir_ind_by_decade %<>%
  mutate(decade = case_when(publicationYear >= 1960 & publicationYear
< 1970 ~ "1970-1979",
                             publicationYear >= 1970 & publicationYear <
1980 ~ "1970-1979",
                             publicationYear >= 1980 & publicationYear <
1990 ~ "1980-1989",
                             publicationYear >= 1990 & publicationYear <
2000 ~ "1990-1999",
                             publicationYear >= 2000 & publicationYear <
2010 ~ "2000-2009",
                             publicationYear >= 2010 & publicationYear <
2020 ~ "2010-2019",
                             publicationYear >= 2020 ~ "2020-2023")
  )

dir_ind_by_decade_lat <- dir_ind_by_decade

dir_ind_by_decade <- dir_ind_by_decade [, c("meso_direct_indirect","decade")]
%>%
  group_by(decade,meso_direct_indirect) %>%
  tally()

colnames(dir_ind_by_decade) <- c("Decade", "Technique", "Count")

unique(dir_ind_by_decade$Technique)

## [1] "indirect" "mixed"

dir_ind_by_decade %<>%
  filter(Technique != "nd" & Technique != "NA") %>% #remove nd from viz becau
se they accounted for a really small percentage
  group_by(Decade) %>%
  mutate(Percentage = round(Count / sum(Count) * 100,1))
```

Make bar plot of technique by decade

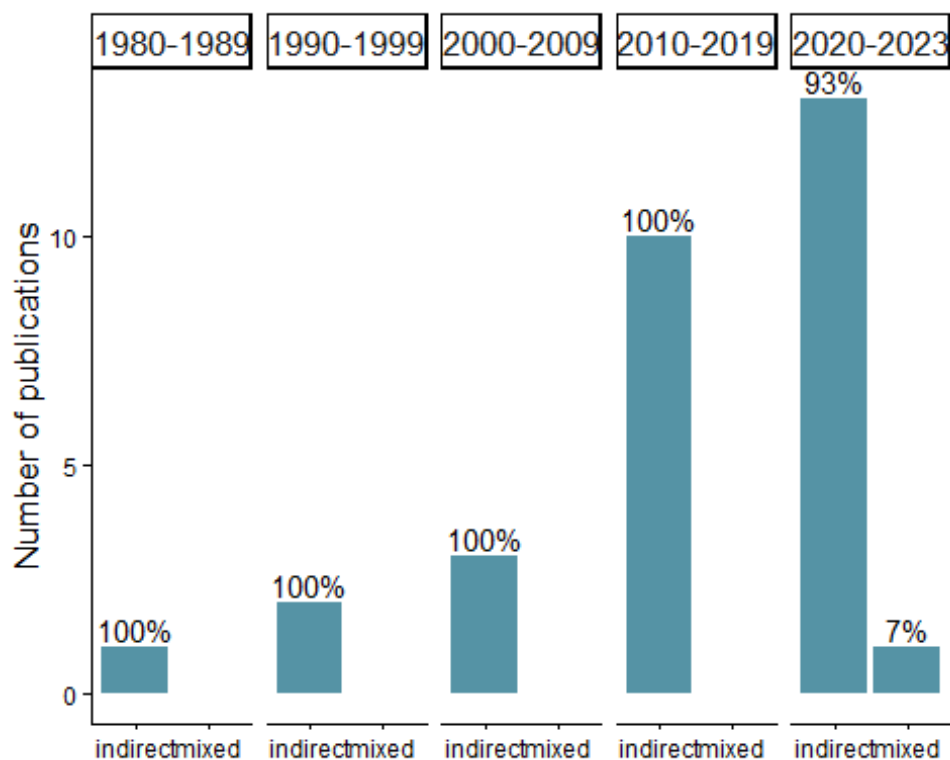
```
technique_decade_barplot <- ggplot(subset(dir_ind_by_decade,
      dir_ind_by_decade$Decade != "1970-1979"),
  aes(x=Technique, y=Count, label = Technique)
  ) +
  geom_bar(position="dodge",
    stat="identity", alpha= 0.8, fill = "#2a788e" ) +
  geom_text(aes(label = paste(round(Percentage,0),"%", sep="")),
    parse = F,
    vjust = -0.25,
```

```

        size=3.9)+
facet_grid(~ Decade,
           switch = "y") +
theme_classic() +
theme(legend.position = "bottom",
      legend.text = element_text(size = 10),
      axis.text.y = element_text(size=8.7),
      axis.title.x = element_blank(),
      axis.title.y = element_text(size=13),
      strip.text = element_text(size = 13))+
labs(y= "Number of publications")

```

technique_decade_barplot



Export bar plot of technique by decade

```

ggsave("technique_decade_barplot.tiff",
       plot = technique_decade_barplot,
       units = "in", width = 7, height = 6, dpi = 300, compression = 'lzw',
       path = here ("Extracted_data_analysis", "Output-Visualization"))

ggsave("technique_decade_barplot.png",
       plot = technique_decade_barplot,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis", "Output-Visualization"))

```

FIELD & INDICATORS

Create subset for field & indicators

```
field_merged <- polar[,c(59:length(polar))]  
  
field_merged[,] <- lapply(field_merged[,], as.numeric)  
  
colSums(field_merged)  
  
##           field_cons           field_commdesc  
##           1           18  
##           field_map           field_impass  
##           11           5  
##           field_troph           field_taxo  
##           5           3  
##           field_dist           field_aqua  
##           4           0  
##           field_method           field_physio  
##           1           1  
##           field_func           field_habtype  
##           5           0  
##           field_fishery           field_stock  
##           2           1  
##           field_paleo           field_phylo  
##           0           0  
##           field_rest           field_archaeo  
##           0           0  
##           field_micro           field_biomedicine  
##           0           0  
##           field_chem           field_Molecular_ecology  
##           0           2  
##           field_Acoustics           field_environmental_drivers  
##           5           13  
##           field_Behaviour           field_Geomorphology  
##           5           2  
##           field_Taxonomy  
##           3
```

Select only the most common fields

```
field_merged <- field_merged[, colSums(field_merged) > 3]  
  
field_merged$field_Taxonomy <- NULL  
  
colSums(field_merged)  
  
##           field_commdesc           field_map  
##           18           11  
##           field_impass           field_troph
```

```
##          5          5
##          field_dist          field_func
##          4          5
##          field_Acoustics field_environmental_drivers
##          5          13
##          field_Behaviour
##          5

field_merged <- field_merged[, order(colSums(field_merged), decreasing = F)]

colSums(field_merged)

##          field_dist          field_impact
##          4          5
##          field_troph          field_func
##          5          5
##          field_Acoustics          field_Behaviour
##          5          5
##          field_map field_environmental_drivers
##          11          13
##          field_commdesc
##          18
```

Rename fields with long version names

```
field_merged %<>%
  rename(`Biodiversity and \n community structure (18)` = field_commdesc,
        `Distribution and \n connectivity (4)` = field_dist,
        `Disturbances (5)` = field_impact,
        `Trophic ecology (4)` = field_troph,
        `Functional ecology (5)` = field_func,
        `Mapping (11)` = field_map,
        `Behaviour (5)` = field_Behaviour ,
        `Acoustics (5)` = field_Acoustics ,
        `Environmental drivers and \n spatio-temporal patterns (13)` = field_
environmental_drivers
  )
colSums(field_merged)

##          Distribution and \n connectivity (4)
##          4
##          Disturbances (5)
##          5
##          Trophic ecology (4)
##          5
##          Functional ecology (5)
##          5
##          Acoustics (5)
##          5
##          Behaviour (5)
##          5
```

```
## Mapping (11)
## 11
## Environmental drivers and \n spatio-temporal patterns (13)
## 13
## Biodiversity and \n community structure (18)
## 18
```

Prepare the indicator matrix

```
ind_matrix <- split_results[["indicators"]]
```

Select only the most common indicators

```
colSums(ind_matrix)

## Abundance Density Diversity Trophic
## 14 11 19 4
## Biomass Behaviour Habitat Size
## 4 4 9 4
## Cover Morphological taxo growth
## 8 7 3 1
## geneticSequences Litter Physiological func
## 2 1 1 1
## Recruitment
## 1

ind_matrix <- ind_matrix[, order(colSums(ind_matrix), decreasing = TRUE)]

ind_matrix <- as.data.frame(ind_matrix)

ind_matrix <- ind_matrix[!purrr::map_lgl(ind_matrix, ~ all(is.na(.)))] # remove empty column
```

Remove taxonomy studies which have a dedicated section

```
ind_matrix$taxo <- NULL
field_merged$field_taxo <- NULL

colSums(ind_matrix)

## Diversity Abundance Density Habitat
## 19 14 11 9
## Cover Morphological Trophic Biomass
## 8 7 4 4
## Behaviour Size geneticSequences growth
## 4 4 2 1
## Litter Physiological func Recruitment
## 1 1 1 1
```

Rename indicators with long version names

```
ind_matrix %<>%
  rename(`Abundance \n (14)` = Abundance,
         `Cover \n (8)` = Cover,
         `Diversity \n (19)` = Diversity,
         `Habitat \n (9)` = Habitat,
         `Morphology \n (7)` = Morphological,
         `Size \n (4)` = Size,
         `Recruitment \n (1)` = Recruitment,
         `Density \n (11)` = Density,
         `Physiological \n indicator \n (1)` = Physiological,
         `Litter \n (1)` = Litter,
         `Biomass \n (4)` = Biomass,
         `Genetic \n sequence \n (2)` = geneticSequences,
         `Functional indicators \n (1)` = func,
         `Growth \n (1)` = growth,
         `Trophic \n (4)` = Trophic,
         `Behaviour \n (4)` = Behaviour
  )
```

```
colSums(ind_matrix)
```

```
##          Diversity \n (19)          Abundance \n (14)
##                      19                      14
##          Density \n (11)          Habitat \n (9)
##                      11                      9
##          Cover \n (8)          Morphology \n (7)
##                      8                      7
##          Trophic \n (4)          Biomass \n (4)
##                      4                      4
##          Behaviour \n (4)          Size \n (4)
##                      4                      4
##          Genetic \n sequence \n (2)          Growth \n (1)
##                      2                      1
##          Litter \n (1) Physiological \n indicator \n (1)
##                      1                      1
##          Functional indicators \n (1)          Recruitment \n (1)
##                      1                      1
```

Multiply the transpose of the field matrix with the indicator matrix

```
co_occurrence_matrix <- t(as.matrix(field_merged)) %*% as.matrix(ind_matrix)
```

Reshape the co-occurrence matrix for plotting}

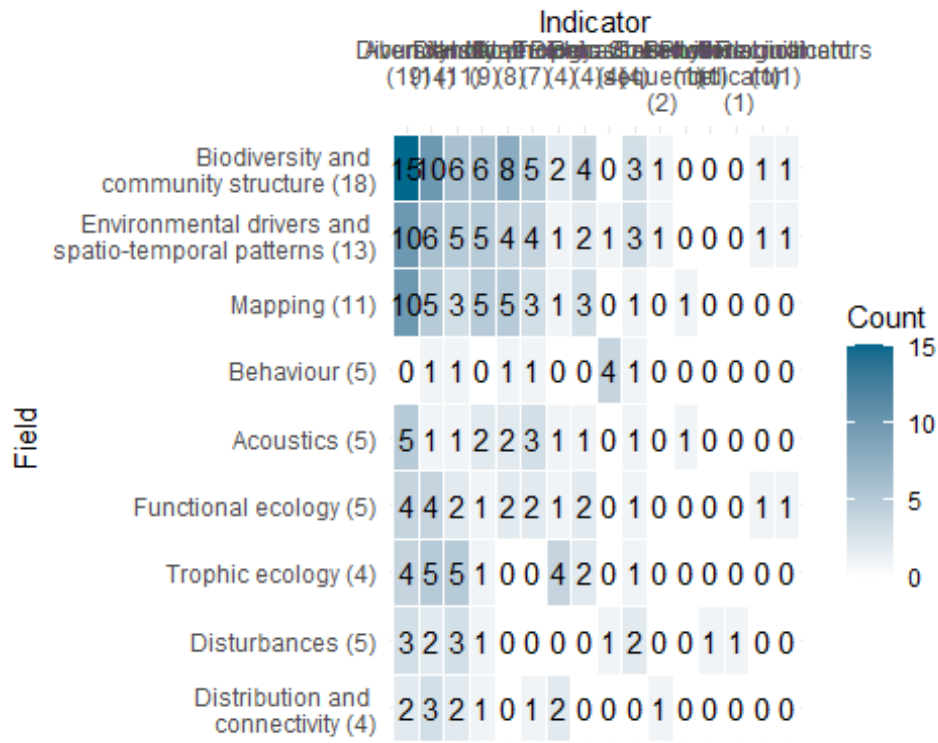
```
co_occurrence_df <- reshape2::melt(co_occurrence_matrix)
```

```
colnames(co_occurrence_df) <- c("Field", "Indicator", "Count")
```

Plot the heatmap

```
field_ind_heat <- ggplot(co_occurrence_df, aes(x = Indicator, y = Field, fill
= Count)) +
  geom_tile(color = "white") +
  scale_fill_gradient(low = "white", high = "deepskyblue4") +
  geom_text(aes(label = Count), color = "black") +
  theme_minimal()+
  scale_x_discrete(position="top")
```

field_ind_heat



Export plot

```
ggsave("field_ind_heat.png",
  units = "in", width = 12.5, height = 5.5,
  path = here ("Extracted_data_analysis", "polar",
  "Output-Visualization"))
```

Calculate # of ecological indicators per record

Create a column with the sum of ecological indicators values to see how many different sampling units were adopted by each study

```
ind_summary <- as.data.frame(split_results[["indicators"]])

ind_summary <- ind_summary[!purrr::map_lgl(ind_summary, ~ all(is.na(.)))] # remove empty column
```

```
ind_summary %<>% rowwise() %>%
  mutate(ind_per_record = sum(c_across(where(is.numeric))))
```

Calculate number of records by number of sampling units in %

```
ind_summary$ind_per_record <- as.factor(ind_summary$ind_per_record)
```

```
ind_per_record <- ind_summary %>%
  group_by(ind_per_record) %>%
  summarise(n_records = length(ind_per_record)) %>%
  mutate(percentage= round(n_records / sum(n_records) * 100,1))
```

```
ind_per_record
```

```
## # A tibble: 6 × 3
##   ind_per_record n_records percentage
##   <fct>          <int>      <dbl>
## 1 1              3         10
## 2 2              5         16.7
## 3 3             14         46.7
## 4 4              2          6.7
## 5 5              5         16.7
## 6 6              1          3.3
```

SAMPLING UNIT

Prepare a data frame for sampling unit

```
sampling_df <- polar %>%
  filter(is.na(taxonomic_publication)) %>% # remove purely taxonomic studies
  select(File,recordTitle,DOI,publicationYear,mininumSurveyedDepthInMeters,maximumSurveyedDepthInMeters,samplingUnit)
```

```
sampling_df %>% tidyr::drop_na(samplingUnit)
sampling_df[!is.na(sampling_df$samplingUnit),]
```

```
sampling_df %<>%
  filter(samplingUnit != "notapp")
```

```
sampling_df$samplingUnit <- as.character(sampling_df$samplingUnit)
```

Prepare columns per sampling units

```
sampling_lvls <- unique(unlist(str_split(sampling_df$samplingUnit,
                                         "\\+"))
                        )
rownames(is.na(sampling_df$samplingUnit))
```

```
## NULL

sampling_df[,sampling_lvls] <- sapply(sampling_lvls,
                                     function(y){
                                       str_count(sampling_df$samplingUnit, y)
                                     })

rm(sampling_lvls)

colnames(sampling_df)

## [1] "File" "recordTitle"
## [3] "DOI" "publicationYear"
## [5] "mininumSurveyedDepthInMeters" "maximumSurveyedDepthInMeters"
## [7] "samplingUnit" "quadrat"
## [9] "nd" "transect"
## [11] "sample" "stationary"
## [13] "expunit"
```

Calculate # of sampling unit per record

Create a column with the sum of sampling units values to see how many different sampling units were adopted by each study

```
sampling_df %<>%
  mutate(type_per_record = rowSums(across(quadrat:expunit)
                                     )
  )
```

Calculate number of records by number of sampling units in %

```
sampling_df$type_per_record <- as.factor(sampling_df$type_per_record)

sampling_unit_per_record <- sampling_df %>%
  group_by(type_per_record) %>%
  summarise(n_records = length(type_per_record)) %>%
  mutate(percentage= round(n_records / sum(n_records) * 100,0))

sampling_unit_per_record

## # A tibble: 2 × 3
##   type_per_record n_records percentage
##   <fct>          <int>      <dbl>
## 1 1              28         93
## 2 2              2          7
```

Create a data frame with number of publication per sampling unit type

```
sampling_unit_tot <- sampling_df %>%
  summarise(across(c(8:(length(sampling_df)-1)),
```

```

        list(sum)))

sampling_unit_tot <- reshape2::melt(sampling_unit_tot)
## No id variables; using all as measure variables
colnames(sampling_unit_tot) <- c("Sampling_unit", "Count")

sampling_unit_tot %<>%
  mutate_at("Sampling_unit",
            stringr::str_replace, "_1", "")

```

Simplified sampling units

Create a column with a simplified sampling units for transect and quadrat

```

sampling_unit_tot %<>%
  mutate(Sampling_unit_macro = case_when(Sampling_unit == 'quadrat' |
    Sampling_unit == "permquad" ~ "Quadrat",
    Sampling_unit == "transect" |
    Sampling_unit == "lit" |
    Sampling_unit == "pit" |
    Sampling_unit == "permlit" |
    Sampling_unit == "permtrans" |
    Sampling_unit == "circular_transect" |
    Sampling_unit == "radial_transect" ~ "Transect",

    Sampling_unit == "nd" ~ "ND",
    Sampling_unit == "stationary" ~ "Stationary",

    Sampling_unit == "expunit" ~ "Experimental",

    Sampling_unit == "sample" ~ "Sample",
    .default = "Other")
  )

# Change columns order
sampling_unit_tot <- cbind(sampling_unit_tot$Sampling_unit_macro,
                          sampling_unit_tot$Sampling_unit,
                          sampling_unit_tot$Count)

colnames(sampling_unit_tot) <- c("Sampling_unit_macro", "Sampling_unit", "Count")

sampling_unit_tot <- as.data.frame(sampling_unit_tot)

sampling_unit_tot[,1:2] <- lapply(sampling_unit_tot[,1:2], as.factor)
sampling_unit_tot$Count <- as.numeric(sampling_unit_tot$Count)

```

```
sampling_unit_simplified <- aggregate(Count ~ Sampling_unit_macro, data = sam
pling_unit_tot, FUN = sum)

sampling_unit_simplified %<>%
  group_by(Sampling_unit_macro) %>%
  mutate(Percentage_macro= round(Count / sum(sampling_unit_simplified$Count)
* 100,1))

sampling_unit_simplified

## # A tibble: 6 × 3
## # Groups:   Sampling_unit_macro [6]
##   Sampling_unit_macro Count Percentage_macro
##   <fct>             <dbl>           <dbl>
## 1 Experimental unit     1             3.1
## 2 ND                   2             6.2
## 3 Quadrat              7            21.9
## 4 Sample               6            18.8
## 5 Stationary           2             6.2
## 6 Transect            14            43.8
```

Build a treemap with simplified sampling unit

```
library(treemapify)

RdYlBu <- RColorBrewer::brewer.pal(7, "RdYlBu")
RdYlBu

## [1] "#D73027" "#FC8D59" "#FEE090" "#FFFFBF" "#E0F3F8" "#91BFDB" "#4575B4"

sampling_simplified_treemap <-
ggplot(sampling_unit_simplified,
  aes(area=Count,
    fill=Sampling_unit_macro,
    line = "black",
    label=paste0(Sampling_unit_macro, " \n ",Percentage_macro,"%"))
  )+
  treemapify::geom_treemap(layout = "squarified")+
  geom_treemap_text(place = "centre",
    size = 20)+
  theme(plot.title = element_text(hjust = 0.5, size= 20),
    legend.position = "none")+
  scale_fill_manual(values= c("#FC8D59", "#FEE090", "#FFFFBF", "#E0F3F8", "#D7
3027", "#91BFDB", "#4575B4"))
  scale_fill_brewer(palette = "RdYlBu")

## <ggproto object: Class ScaleDiscrete, Scale, gg>
##   aesthetics: fill
##   axis_order: function
##   break_info: function
##   break_positions: function
```

```

##      breaks: waiver
##      call: call
##      clone: function
##      dimension: function
##      drop: TRUE
##      expand: waiver
##      fallback_palette: function
##      get_breaks: function
##      get_breaks_minor: function
##      get_labels: function
##      get_limits: function
##      get_transformation: function
##      guide: legend
##      is_discrete: function
##      is_empty: function
##      labels: waiver
##      limits: NULL
##      make_sec_title: function
##      make_title: function
##      map: function
##      map_df: function
##      minor_breaks: waiver
##      n.breaks.cache: NULL
##      na.translate: TRUE
##      na.value: NA
##      name: waiver
##      palette: function
##      palette.cache: NULL
##      position: left
##      range: environment
##      rescale: function
##      reset: function
##      train: function
##      train_df: function
##      transform: function
##      transform_df: function
##      super: <ggproto object: Class ScaleDiscrete, Scale, gg>

sampling_unit_simplified %<>%
  mutate(case_when(Sampling_unit_macro == "Experimental unit" ~
    "Experimental \n unit", T ~ Sampling_unit_macro))

sampling_simplified_treemap <-
ggplot(sampling_unit_simplified,
  aes(area=Count,
    fill= reorder(Sampling_unit_macro, -Count),
    line = "black",
    label=paste0(Sampling_unit_macro, " \n ", Percentage_macro, "%"))
  )+
  treemapify::geom_treemap(layout = "squarified")+

```


Quadrat extracted from transect

```
print(paste("Number of publications where quadrat is the sampling unit or one
of the sampling unit: ",
           nrow(quad_design <- subset(polar, grepl("quad", samplingUnit)
                                     )
           )
)

## [1] "Number of publications where quadrat is the sampling unit or one of t
he sampling unit: 7"

print(paste("Number of publications where quadrats were collected along trans
ects: ", nrow(subset(quad_design,
                    grepl("transect\\(quadrat", meso_design) |
                    grepl("transect\\(depth\\(quadrat", meso_design)
                    )
)
)
)

## [1] "Number of publications where quadrats were collected along transects:
5"
```

DATA TYPE

```
data_type <- count_results[["collectedDataType"]]

data_type$Percentage <- round(data_type$Percentage, 1)

data_type

## # A tibble: 10 × 3
##   collectedDataType      Count Percentage
##   <fct>              <int>      <dbl>
## 1 photo                4        13.3
## 2 photo+specimen       2         6.7
## 3 photo+specimen+video+water 1         3.3
## 4 photo+video          5        16.7
## 5 sediment+video       1         3.3
## 6 specimen             2         6.7
## 7 specimen+sediment+water 1         3.3
## 8 specimen+video       3         10
## 9 specimen+video+water  1         3.3
## 10 video              10        33.3
```

Lump uncommon combination into “Other”

```

data_type_simplified <- data_type %>%
  rename(`Data type` = collectedDataType) %>%
  mutate(`Data type` = as.character(`Data type`)) %>%
  filter(Percentage > 2) %>%
  bind_rows(
    data_type %>%
      filter(Percentage <= 2) %>%
      summarise(
        `Data type` = "other",
        Count = sum(.data$Count),
        Percentage = sum(.data$Percentage)
      )
  )

data_type_simplified %<>%
  mutate(`Data type` =
    case_when(`Data type` == 'photo' ~ "Photo",
              `Data type` == "video" ~ "Video",
              `Data type` == "other" ~ "Other",
              `Data type` == "specimen" ~ "Specimen",
              `Data type` == "photo+specimen" ~ "Photo + \n Specimen",
              `Data type` == "photo+specimen+video+water" ~ "Photo + \n Specimen + \n Video + \n Water",
              `Data type` == "photo+video" ~ "Photo + \n Video",
              `Data type` == "sediment+video" ~ "Sediment + \n Video",
              `Data type` == "specimen+sediment+water" ~ "Specimen + \n Sediment + \n Water",
              `Data type` == "specimen+video" ~ "Specimen + \n Video",
              `Data type` == "specimen+video+water" ~ "Specimen + \n Water + \n Specimen",
              .default = "Other")
  )

```

Build a treemap with data type

```

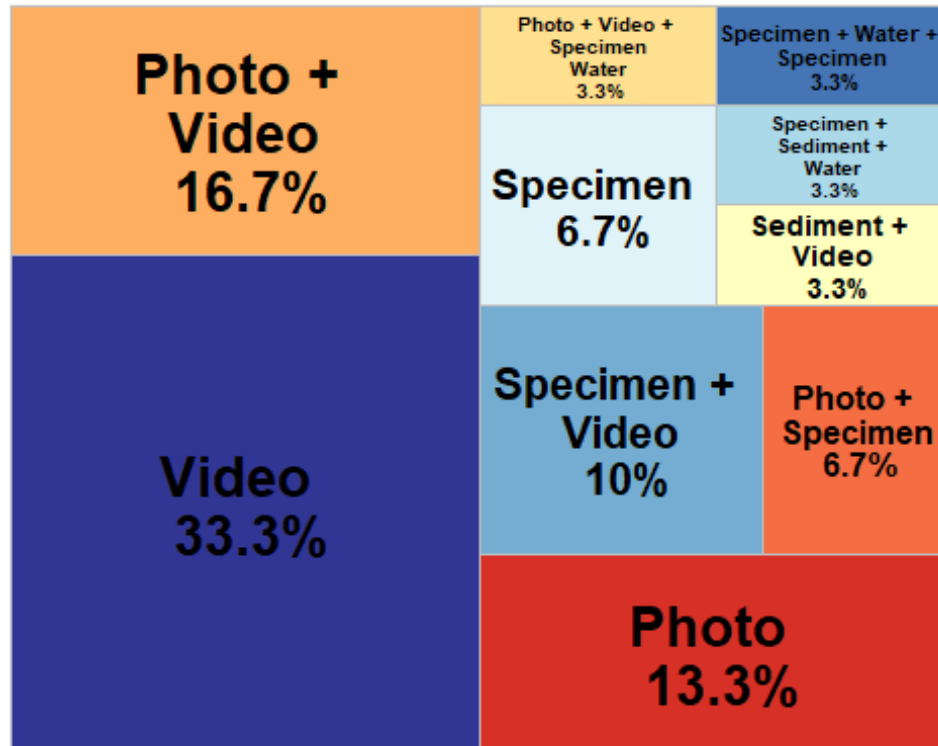
library(treemapify)
library(RColorBrewer)

data_type_treemap <-
ggplot(data_type_simplified,
  aes(area=Count,
    fill=`Data type`,
    line = "black",
    label=paste0(`Data type`, " \n ", Percentage, "%"))
  )+
  treemapify::geom_treemap(layout = "squarified")+

```

```
geom_treemap_text(place = "centre", fontface = "bold",
                  size = 20)+
  theme(plot.title = element_text(hjust = 0.5, size= 20),
        legend.position = "none")+
  scale_fill_brewer(palette = "RdYlBu")
```

data_type_treemap



Export sampling_simplified_treemap

```
ggsave("data_type_treemap.png",
       plot = data_type_treemap,
       units = "in", width = 7, height = 6,
       path = here ("Extracted_data_analysis",
                    "Output-Visualization"))
```

HABIT

```
habit <- as.data.frame(count_results["habit"])

# Rename columns
habit %<>%
  rename(Habit = habit.habit,
         Count = habit.Count,
         Percentage = habit.Percentage)
```

Make the habit donut plot

```
habit %<>%
  mutate(ymax = cumsum(Count),
         ymin = c(0, head(ymax, n = -1)),
         mid = (ymin + ymax) / 2,
         angle = 90 - (mid / sum(Count)) * 360
  )

# Split and assign labels
habit_inside <- habit %>%
  mutate(label = paste0(round(Percentage,1), "%"))

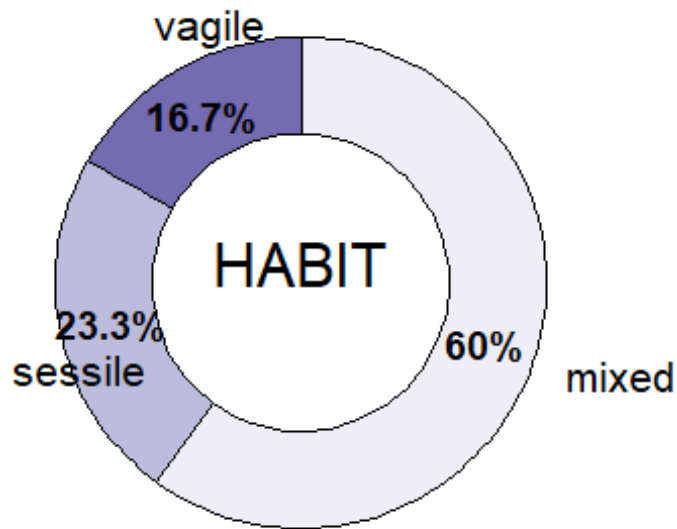
habit_outside <- habit %>%
  mutate(label = Habit)

# Plot
habit_donut <- ggplot() +
  # Donut segments
  geom_rect(data = habit,
            aes(ymin = ymin, ymax = ymax, xmin = 2.5, xmax = 4, fill = Habit),
            color = "black") +

  # Inside labels
  geom_text(data = habit_inside,
            aes(x = 3.2, y = mid, label = label),
            size = 5, color = "black",
            fontface = "bold") +

  # Outside labels with Habit name
  ggrepel::geom_text_repel(data = habit_outside,
                           aes(x = 4.3, y = mid, label = label),
                           size = 5.5,
                           nudge_x = 0.5,
                           direction = "y",
                           segment.color = "black",
                           hjust = 0,
                           box.padding = 0.3,
                           max.overlaps = Inf) +
  coord_polar(theta = "y") +
  xlim(0.2, 5) +
  scale_fill_brewer(palette = "Purples") +
  theme_void() +
  theme(legend.position = "none")+
  annotate(geom = 'text', x = 0.5, y = 0,
          label = c("HABIT"), size = 8)

habit_donut
```



Export habit_donut

```
ggsave("habit_donut.png",
  plot = habit_donut,
  units = "in", width = 7, height = 6,
  path = here ("Extracted_data_analysis",
               "Output-Visualization"))
```

SUBSTRATE

```
substrate <- as.data.frame(count_results["meso_substrate"])

# Rename columns
substrate %<>%
  rename(Substrate = meso_substrate.meso_substrate,
         Count = meso_substrate.Count,
         Percentage = meso_substrate.Percentage)

# Remove rows with nd
substrate <- substrate[!substrate$Substrate %in% "nd",]

# Merge substrate combination with less than 3% and recalculate Count and Percentages
substrate <- substrate %>%
  mutate(Substrate = if_else(Percentage < 3, "Other combinations", Substrate))
%>%
```

```

group_by(Substrate) %>%
select(-Percentage) %>%
summarise(
  Count = sum(Count)) %>%
mutate(Percentage = round(Count / sum(Count) * 100,1)
)

substrate %<>%
mutate(
  Substrate = case_when(Substrate == "hard" ~ "Hard",
                        Substrate == "soft" ~ "Soft",
                        Substrate == "mixed" ~ paste0("Hard+", "\n", "Soft"),
                        Substrate == "artificial" ~ "Artificial",
                        Substrate == "Other combinations" ~ "Other combinat
ions"))

```

Make the substrate donut plot

```

substrate %<>%
mutate(
  ymax = cumsum(Count),
  ymin = c(0, head(ymax, n = -1)),
  mid = (ymin + ymax) / 2,
  angle = 90 - (mid / sum(Count)) * 360
)

# Split and assign labels
substrate_inside <- substrate %>%
  filter(Percentage >= 5) %>%
  mutate(label = paste0(Percentage, "%"))

substrate_outside <- substrate %>%
  mutate(label = Substrate)

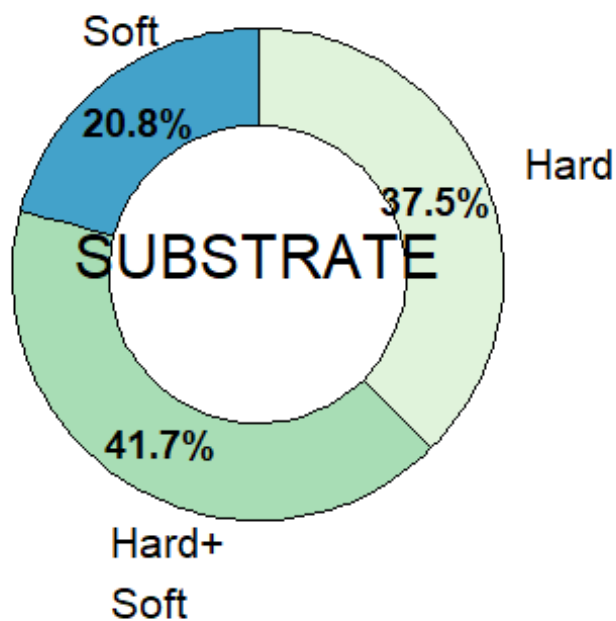
# Plot
substrate_donut <- ggplot() +
  # Donut segments
  geom_rect(data = substrate,
            aes(ymin = ymin, ymax = ymax, xmin = 2.5, xmax = 4, fill = Substr
ate),
            color = "black") +

  # Inside labels
  geom_text(data = substrate_inside,
            aes(x = 3.2, y = mid, label = label),
            size = 5, color = "black",
            fontface = "bold") +

```

```
# Outside Labels with substrate name
ggrepel::geom_text_repel(data = substrate_outside,
  aes(x = 4.4, y = mid, label = label),
  size = 5.5,
  nudge_x = 0.3,
  direction = "y",
  segment.color = "black",
  hjust = 0,
  box.padding = 0.3,
  max.overlaps = Inf) +
coord_polar(theta = "y") +
xlim(0.2, 5) +
scale_fill_brewer(palette = "GnBu") +
theme_void() +
theme(legend.position = "none")+
annotate(geom = 'text', x = 0.5, y = 0,
  label = c("SUBSTRATE"), size = 8)
```

substrate_donut



Export the

substrate donut plot

```
ggsave("substrate_donut.png",
  plot = substrate_donut,
  units = "in", width = 7, height = 6,
  path = here ("Extracted_data_analysis",
    "Output-Visualization"))
```

BENTHIC POSITION

Create a data frame with benthic position

```
position <- as.data.frame(count_results["environmentalPosition"])

# Rename columns
position %<>%
  rename(Position = environmentalPosition.environmentalPosition,
         Count = environmentalPosition.Count,
         Percentage = environmentalPosition.Percentage)

# Merge position combination with less than 3% and recalculate Count and Percentages
position <- position %>%
  mutate(Position = if_else(Percentage < 4, "Other combinations", Position))
  %>%
  group_by(Position) %>%
  select(-Percentage) %>%
  summarise(
    Count = sum(Count)) %>%
  mutate(Percentage = round(Count / sum(Count) * 100, 1)
  )
```

Make the benthic position donut plot

```
position %<>%
  mutate(
    Position = case_when(Position == "epi" ~ "Epi",
                        Position == "endo" ~ "Endo",
                        Position == "dem+epi" ~ "Hyper + \n Epi",
                        Position == "endo+epi" ~ "Endo + \n Epi",
                        Position == "Other combinations" ~ "Other combinations"),
    ymax = cumsum(Count),
    ymin = c(0, head(ymax, n = -1)),
    mid = (ymin + ymax) / 2,
    angle = 90 - (mid / sum(Count)) * 360
  )

# Split and assign labels
position_inside <- position %>%
  filter(Percentage > 3) %>%
  mutate(label = paste0(Percentage, "%"))

position_outside <- position %>%
  mutate(label = Position)
```

```

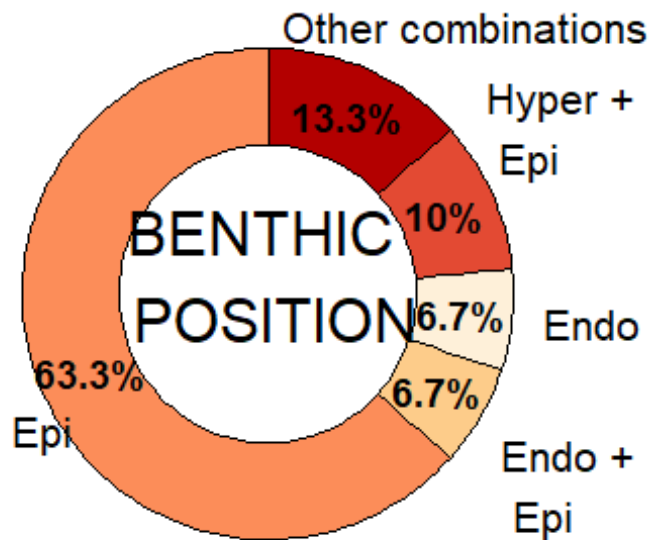
# Plot
benthic_position_donut <- ggplot() +
  # Donut segments
  geom_rect(data = position,
            aes(ymin = ymin, ymax = ymax, xmin = 2.5, xmax = 4, fill = Positi
on),
            color = "black") +

  # Inside labels
  geom_text(data = position_inside,
            aes(x = 3.2, y = mid, label = label),
            size = 5, color = "black",
            fontface = "bold") +

  # Outside labels with Position name
  ggrepel::geom_text_repel(data = position_outside,
                           aes(x = 4.2, y = mid, label = label),
                           size = 5.5,
                           nudge_x = 0.3,
                           direction = "y",
                           segment.color = "black",
                           hjust = 0,
                           box.padding = 0.3,
                           max.overlaps = Inf) +
  coord_polar(theta = "y") +
  xlim(0.2, 5) +
  scale_fill_brewer(palette = "OrRd") +
  theme_void() +
  theme(legend.position = "none")+
  annotate(geom = 'text', x = 0.5, y = 0,
          label = c("BENTHIC \n POSITION"), size = 8)

benthic_position_donut

```



Export the benthic_position donut plot

```
ggsave("benthic_position_donut.png",
  plot = benthic_position_donut,
  units = "in", width = 7, height = 6,
  path = here ("Extracted_data_analysis",
               "Output-Visualization"))
```

TARGET

Target level

```
target_level <- as.data.frame(count_results["taxonomicTargetLevel"])

# Rename columns
target_level %<>%
  rename(Level = taxonomicTargetLevel.taxonomicTargetLevel,
         Count = taxonomicTargetLevel.Count,
         Percentage = taxonomicTargetLevel.Percentage)

# Remove rows with nd
target_level <- target_level[!target_level$Level %in% "species,species",]
target_level <- target_level[!target_level$Level %in% ",species",]

# Merge habit combination with less than 3% and recalculate Count and Percent
ages
```

```

target_level <- target_level %>%
  mutate(Level = if_else(Percentage < 3, "Other combinations", Level)) %>%
  group_by(Level) %>%
  select(-Percentage) %>%
  summarise(
    Count = sum(Count)) %>%
  mutate(Percentage = round(Count / sum(Count) * 100,1)
  )

```

Treemap with the taxonomic target level

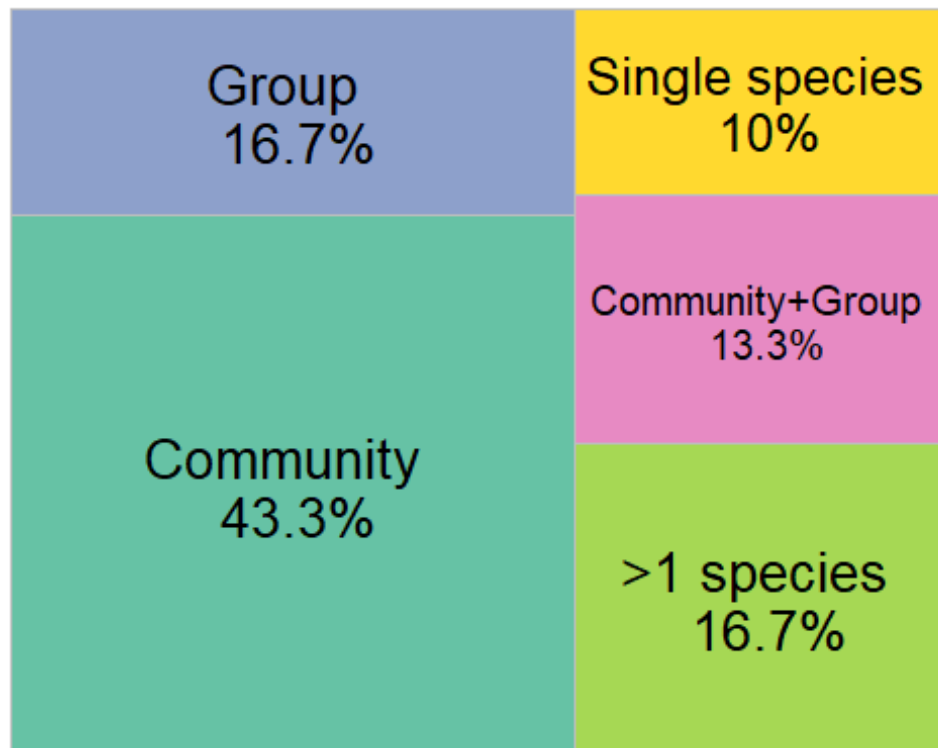
```

target_level_labels <- target_level %<>%
  mutate(
    Level = case_when(Level == "comm" ~ "Community",
                      Level == "group" ~ "Group",
                      Level == "single" ~ "Single species",
                      Level == "species" ~ ">1 species",
                      Level == "comm+group" ~ "Community+Group",
                      Level == "comm+species" ~ "Community+Species",
                      Level == "group+species" ~ "Group+\nSpecies",
                      Level == "Other combinations" ~ "Other")
  )

target_level_treemap <- ggplot(target_level_labels,
  aes(area=Count,
    fill=Level,
    line = "black",
    label=paste0(Level,
      " \n ", Percentage, "%"))
  )+
  treemapify::geom_treemap(layout = "squarified")+
  treemapify::geom_treemap_text(place = "centre",
    size = 20)+
  theme(plot.title = element_text(hjust = 0.5, size= 20),
    legend.position = "none")+
  scale_fill_manual(values= c("#A6D854" , "#66C2A5", "#E78AC3", "#8DA0CB", "#FFD92F", "#B3B3B3", "#E5C494", "#FC8D62"))

target_level_treemap

```



```
ggsave("target_level_treemap.png",
  plot = target_level_treemap,
  units = "in", width = 9, height = 6,
  path = here ("Extracted_data_analysis",
               "Output-Visualization"))
```

"polar",

Detected phyla in community studies

```
detected <- polar %>%
  filter(str_detect(taxonomicTargetLevel, "comm")) %>%
  select(File,
    detected_meso_cnidaria,
    detected_meso_sponges,
    detected_meso_seagrass,
    detected_meso_algae,
    detected_meso_mollusca,
    detected_meso_chondrichthyes,
    detected_meso_fish,
    detected_meso_crustacea,
    detected_meso_echino,
    detected_meso_other_taxa)

detected_lvls <-
  unique(
    unlist(
      str_split(detected$detected_meso_other_taxa, "\\+")
    )
  )
```

```

    )
detected_lvls<- detected_lvls[! detected_lvls %in% c("",NA,"notapp","", "nd")]
detected[,detected_lvls] <- sapply(detected_lvls,
                                   function(y){
                                     str_count(detected$detected_meso_othe
r_taxa, y)
                                   }
                                   )

rm(detected_lvls)

detected %<>%
  mutate(across(everything(), as.character)) %>%
  replace(is.na(.), "0") %>%
  select(-detected_meso_other_taxa)

detected <- detected %>%
  mutate(across(-File, ~ case_when(
    .x %in% c("x", "x,", "xx") ~ "1",
    .x == ",", ~ "0",
    TRUE ~ .x
  ))) %>%
  mutate(across(-File, as.numeric))

```

Calculate number of publications per taxonomic group

```

detected_tot <-
  detected %>%
  summarise(across(where(is.numeric),
                    list(sum)
                  )
            )

detected_tot <- reshape2::melt(detected_tot, id.vars = NULL)

colnames(detected_tot) <- c("Taxa", "Count")

detected_tot %<>%
  mutate_at("Taxa",
            stringr::str_replace, "_1", "") %>%
  mutate_at("Taxa",
            stringr::str_replace, "detected_meso_", "")

detected_tot %<>%
  mutate(Percentage= round(Count / nrow(detected) * 100,1))

rm(detected)

```

Remove absent taxonomic groups

```
detected_tot_collapsed <- subset(detected_tot, detected_tot$Count>0)
```

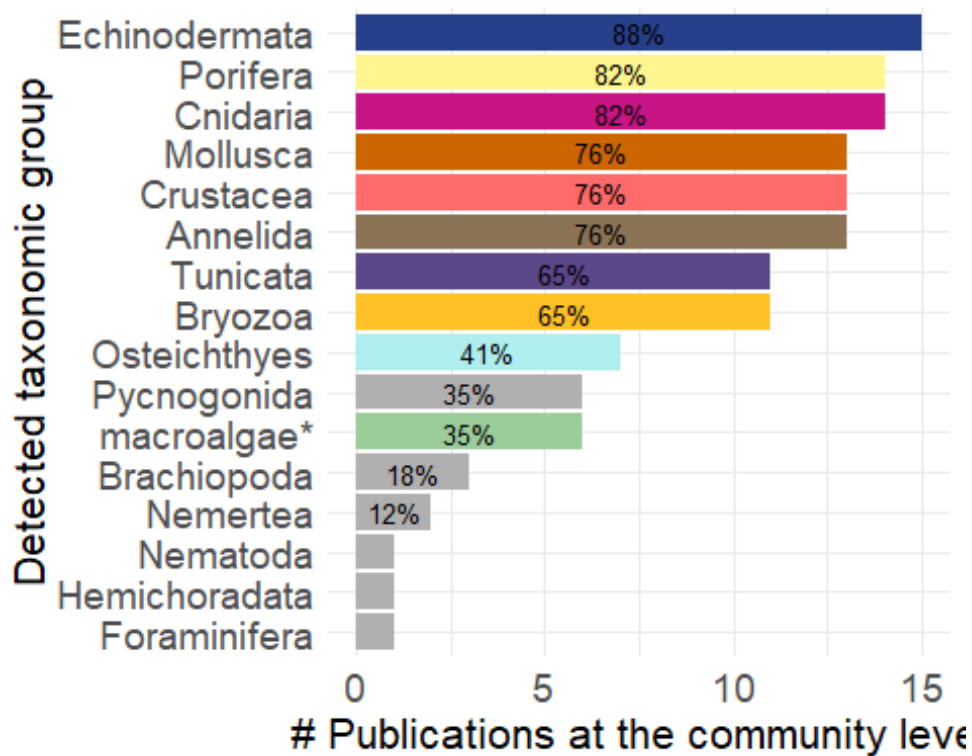
Create a bar chart with detected taxonomic groups

```
detected_bar <- ggplot(detected_tot_collapsed, aes(x = reorder(Taxa, Count),
y = Count, fill= Taxa)) +
  geom_bar(stat = "identity") +
  geom_text(
    data = detected_tot_collapsed %>% filter(Percentage > 11),
    aes(label = paste0(round(Percentage, 0), "%"),
    position = position_stack(vjust = 0.5),
    color = "black",
    size = 3.5
  )+
  labs(y = "# Publications at the community level", x = "Detected taxonomic group") +
  theme_minimal() +
  coord_flip()+ # optional: flips for horizontal bars
  theme(legend.position = "none") +
  scale_x_discrete(labels = rev(c("Echinodermata",
    "Porifera",
    "Cnidaria",
    "Mollusca",
    "Crustacea",
    "Annelida",
    "Tunicata",
    "Bryozoa",
    "Osteichthyes",
    "Pycnogonida",
    "macroalgae*",
    "Brachiopoda",
    "Nemertea",
    "Nematoda",
    "Hemichordata",
    "Foraminifera"))))

detected_palette <- c("algae" = "darkseagreen3",
  "cnidaria" = "mediumvioletred",
  "crustacea" = "indianred1",
  "sponges" = "khaki1",
  "mollusca" = "darkorange3",
  "fish" = "paleturquoise2",
  "echino" = "royalblue4",
  "Tunicata" = "mediumpurple4",
  "Hemichordata" = "gray69",
  "Brachiopoda" = "gray69",
  "Nemertea" = "gray69",
  "Pycnogonida" = "gray69",
```

```
"Nematoda" = "gray69",
"Foraminifera" = "gray69",
"Bryozoa" = "goldenrod1",
"Annelida" = "burlywood4")
```

```
detected_bar +
  scale_fill_manual(values = detected_palette,
    limits = names(detected_palette))+
  theme(axis.text = element_text(size = 14),
    axis.title = element_text(size = 16)
  )
```



```
ggsave("detected_bar.png",
  units = "in", width = 7, height = 6,
  path = here ("Extracted_data_analysis",
    "Output-Visualization"))
```

"polar",

Target taxonomic groups not at the community level

```
group <- polar %>%
  filter(taxonomicTargetLevel != "comm") %>%
  select(File,
    detected_meso_cnidaria,
    detected_meso_sponges,
    detected_meso_seagrass,
```

```

    detected_meso_algae,
    detected_meso_mollusca,
    detected_meso_chondrichthyes,
    detected_meso_fish,
    detected_meso_crustacea,
    detected_meso_echino,
    detected_meso_other_taxa,
    targetTaxa)

group_lvls <-
  unique(
    unlist(
      str_split(group$detected_meso_other_taxa, "\\+")
    )
  )
group_lvls<- group_lvls[! group_lvls %in% c("",NA,"notapp",",","nd")]
group[,group_lvls] <- sapply(group_lvls,
                             function(y){
                               str_count(group$detected_meso_other_t
axa, y)
                             }
                             )

rm(group_lvls)

group <- group %>%
  mutate(across(everything(), as.character)) %>%
  replace(is.na(.), "0") %>%
  select(-detected_meso_other_taxa,-targetTaxa) %>%
  mutate(across(-File, ~ case_when(
    .x %in% c("x", "x,", "xx") ~ "1",
    .x == ",", ~ "0",
    TRUE ~ .x
  ))) %>%
  mutate(across(-File, as.numeric))

group_tot <-
  group %>%
  summarise(across(where(is.numeric),
                    list(sum)
                  )
            )

group_tot <- reshape2::melt(group_tot, id.vars = NULL)

colnames(group_tot) <- c("Taxa", "Count")

group_tot %<>%
  mutate_at("Taxa",
            stringr::str_replace, "_1", "") %>%

```

```

    mutate_at("Taxa",
              stringr::str_replace, "detected_meso_", "")

group_tot %<>%
  mutate(Percentage= round(Count / nrow(group) * 100,1))

group_tot_collapsed <- group_tot %>%
  filter(Percentage > 0 ) %>%
  group_by(Taxa) %>%
  summarise(
    Count = sum(Count),
    Percentage = sum(Percentage),
    .groups = "drop"
  )

group_tot_collapsed %<>%
  mutate(Taxa = case_when(Taxa == 'echino' ~ "Echinodermata",
                          Taxa == 'fish' ~ "Osteichthyes",
                          Taxa == 'mollusca' ~ "Mollusca",
                          Taxa == 'algae' ~ "macroalgae*",
                          Taxa == "sponges" ~ "Porifera",
                          Taxa == "crustacea" ~ "Crustacea",
                          Taxa == "cnidaria" ~ "Cnidaria",
                          Taxa == "algae" ~ "algae*",
                          .default = Taxa))

group_bar <- ggplot(group_tot_collapsed, aes(x = reorder(Taxa, Count), y = Count, fill= Taxa)) +
  geom_bar(stat = "identity") +
  geom_text(
    data = group_tot_collapsed %>% filter(Percentage > 10),
    aes(label = paste0(round(Percentage, 0), "%"),
        position = position_stack(vjust = 0.5),
        color = "black",
        size = 3.5
    )+
  labs(y = "N of publications not at community level", x = "Taxonomic group")
+
  theme_minimal() +
  coord_flip()+ # optional: flips for horizontal bars
  theme(legend.position = "none")

group_palette <- c("macroalgae*" = "darkseagreen3",
                  "Cnidaria" = "mediumvioletred",
                  "Crustacea" = "indianred1",
                  "Porifera" = "khaki1",
                  "Mollusca" = "darkorange3",
                  "Osteichthyes" = "paleturquoise2",
                  "Echinodermata" = "royalblue4",

```

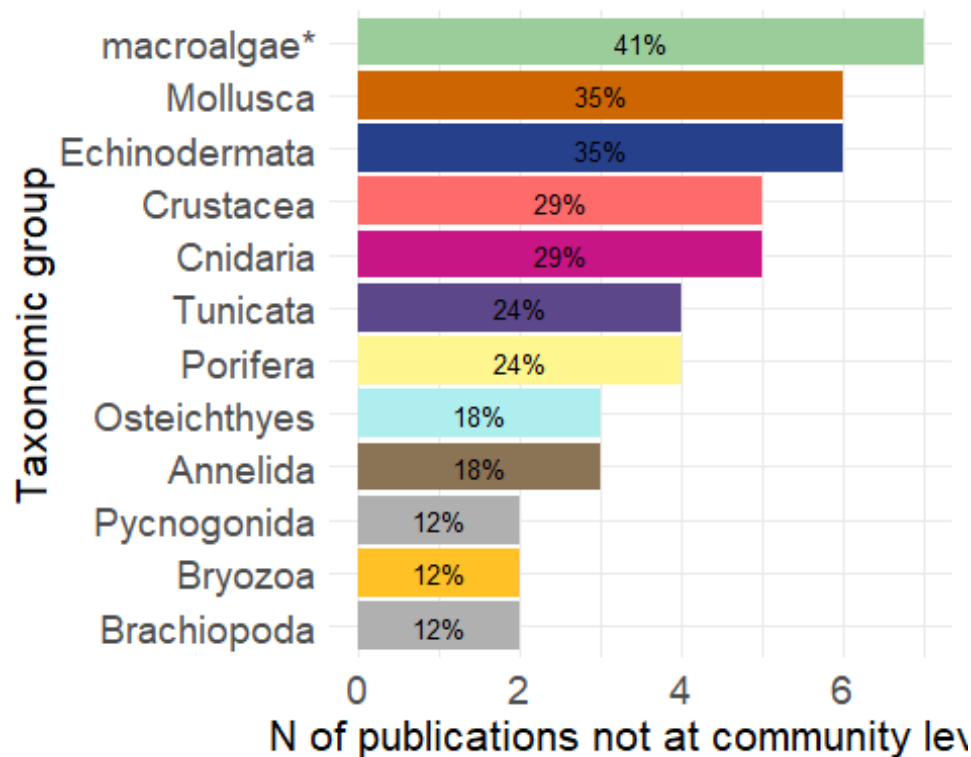
```

    "Tunicata" = "mediumpurple4",
    "Chondrichthyes" = "gray69",
    "Brachiopoda" = "gray69",
    "Pycnogonida" = "gray69",
    "Bryozoa" = "goldenrod1",
    "Annelida" = "burlywood4")

group_bar <- group_bar +
  scale_fill_manual(values = group_palette,
    limits = names(group_palette))+
  theme(axis.text = element_text(size = 14),
    axis.title = element_text(size = 16)
  )

```

group_bar



Export bar plot

```

ggsave("group_bar.png",
  plot = group_bar,
  units = "in", width = 7, height = 6,
  path = here ("Extracted_data_analysis",
    "Output-Visualization"))

```

New species described per phyla

```

new_spec <- polar %>%
  filter(str_detect(taxonomic_publication, "taxa"))

```

```
new_spec %<>%
  select(File,
    detected_meso_cnidaria,
    detected_meso_sponges,
    detected_meso_seagrass,
    detected_meso_algae,
    detected_meso_mollusca,
    detected_meso_chondrichthyes,
    detected_meso_fish,
    detected_meso_crustacea,
    detected_meso_echino,
    detected_meso_other_taxa)
```

DATA CROSSING

```
table(polar$meso_protection)

##
##  nd  no yes
##  26   2   2
```

VISUALLY SURVEYED SURFACE

```
polar$visuallySurveyedSurface <- as.factor(polar$visuallySurveyedSurface)
summary(subset(polar, is.na(polar$taxonomic_publication))$visuallySurveyedSurface)

##      <1      >10      >100      >1000      >10000      >100000      >1000000      nd
##       1         2         4         6         1         1         1       13
## notapp
##       1
```

Observation

```
print(paste("Number of manipulative experimental studies:", nrow(subset(polar,
  grepl("exp", polar$observationalORexperimental)))))

## [1] "Number of manipulative experimental studies: 1"

print(paste("Number of manipulative experimental studies in the field of restoration:", nrow(subset(polar, grepl("exp", polar$observationalORexperimental)
  & polar$field_rest != 0)

  )
)
)

## [1] "Number of manipulative experimental studies in the field of restoration: 0"
```

Temp replicates & exp

```
print(paste("# Publications that included temporal replicates:", nrow(subset(
  polar, grepl("yes", temporalReplicates)))))

## [1] "# Publications that included temporal replicates: 6"

print(paste("# Publications relying on a manipulative experiment that include
  d temporal replicates:", nrow(subset(polar, (grepl("exp", polar$observational
  ORexperimental) & grepl("yes", temporalReplicates)))))

## [1] "# Publications relying on a manipulative experiment that included tem
  poral replicates: 1"
```

ROV & sample

```
rov <- subset(polar, grepl("rov", meso_samplingPlatform) )

rov_specimen <- subset(rovs, grepl("specimen", collectedDataType) )

print(paste("Percentage of publications relying solely on ROV where specimens
  were collected:", round(nrow(rovs_specimen)/nrow(rovs)*100,1), "%"))

## [1] "Percentage of publications relying solely on ROV where specimens were
  collected: 29.4 %"

rm(rovs, rovs_specimen)
```

Molecular techniques in taxonomic and non-taxonomic studies

```
polar$taxonomic_publication <- as.factor(polar$taxonomic_publication )

molecular <- polar %>%
  filter(molecularAnalyses != "no" &
    molecularAnalyses != "notapp" &
    molecularAnalyses != "nd")

print(paste("# Publications that adopted molecular techniques: ", nrow(molecul
  ar)))

## [1] "# Publications that adopted molecular techniques: 2"

molecular_notaxa <- molecular %>%
  filter(is.na(taxonomic_publication))

print(paste("# Publications that adopted molecular techniques not for taxonom
  y: ", nrow(molecular_notaxa)))

## [1] "# Publications that adopted molecular techniques not for taxonomy: 2
  "

rm(molecular, molecular_notaxa)
```

SESSION INFORMATION AND REFERENCES

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Italian_Italy.utf8  LC_CTYPE=Italian_Italy.utf8
## [3] LC_MONETARY=Italian_Italy.utf8 LC_NUMERIC=C
## [5] LC_TIME=Italian_Italy.utf8
##
## time zone: Europe/Rome
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices utils      datasets  methods   base
##
## other attached packages:
## [1] RColorBrewer_1.1-3 treemapify_2.6.0  openxlsx_4.2.8.1  tibble_3.3.1
##
## [5] purrr_1.2.1      stringr_1.6.0      magrittr_2.0.4     readxl_1.4.5
##
## [9] ggplot2_4.0.2     dplyr_1.2.0        here_1.0.2
##
## loaded via a namespace (and not attached):
## [1] utf8_1.2.6        generics_0.1.4     tidyr_1.3.2        stringi_1.8.7
##
## [5] digest_0.6.39     evaluate_1.0.5     grid_4.5.2         fastmap_1.2.0
##
## [9] cellranger_1.1.0  rprojroot_2.1.1    plyr_1.8.9         ggrepel_0.9.6
##
## [13] zip_2.3.3         scales_1.4.0       textshaping_1.0.4  cli_3.6.5
##
## [17] rlang_1.1.7       withr_3.0.2        yaml_2.3.12        otel_0.2.0
##
## [21] tools_4.5.2       reshape2_1.4.5     forcats_1.0.1      vctrs_0.7.1
##
## [25] R6_2.6.1          lifecycle_1.0.5    ragg_1.5.0         pkgconfig_2.0.3
##
## [29] pillar_1.11.1     gtable_0.3.6       glue_1.8.0         Rcpp_1.1.1
##
## [33] ggfittext_0.10.3  systemfonts_1.3.1  xfun_0.56          tidyselect_1.2.
1
## [37] rstudioapi_0.18.0 knitr_1.51         farver_2.1.2       htmltools_0.5.9
```

```

## [41] rmarkdown_2.30      svglite_2.2.2      labeling_0.4.3      compiler_4.5.2

## [45] S7_0.2.1

library(report)
report::report(sessionInfo())

## Analyses were conducted using the R Statistical language (version 4.5.2; R
## Core
## Team, 2025) on Windows 11 x64 (build 26200), using the packages magrittr
## (version 2.0.4; Bache S, Wickham H, 2025), report (version 0.6.3; Makowski
## D et
## al., 2023), here (version 1.0.2; Müller K, 2025), tibble (version 3.3.1; M
## üller
## K, Wickham H, 2026), RColorBrewer (version 1.1.3; Neuwirth E, 2022), openx
## lxx
## (version 4.2.8.1; Schauburger P, Walker A, 2025), ggplot2 (version 4.0.2;
## Wickham H, 2016), stringr (version 1.6.0; Wickham H, 2025), readxl (versio
## n
## 1.4.5; Wickham H, Bryan J, 2025), dplyr (version 1.2.0; Wickham H et al.,
## 2026), purrr (version 1.2.1; Wickham H, Henry L, 2026) and treemapify (ver
## sion
## 2.6.0; Wilkins D, 2025).
##
## References
## -----
## - Bache S, Wickham H (2025). _magrittr: A Forward-Pipe Operator for R_.
## doi:10.32614/CRAN.package.magrittr
## <https://doi.org/10.32614/CRAN.package.magrittr>, R package version 2.0.4,
## <https://CRAN.R-project.org/package=magrittr>.
## - Makowski D, Lüdecke D, Patil I, Thériault R, Ben-Shachar M, Wiernik B
## (2023).
## "Automated Results Reporting as a Practical Tool to Improve Reproducibilit
## y and
## Methodological Best Practices Adoption." _CRAN_.
## doi:10.32614/CRAN.package.report
## <https://doi.org/10.32614/CRAN.package.report>,
## <https://easystats.github.io/report/>.
## - Müller K (2025). _here: A Simpler Way to Find Your Files_.
## doi:10.32614/CRAN.package.here <https://doi.org/10.32614/CRAN.package.her
## e>, R
## package version 1.0.2, <https://CRAN.R-project.org/package=here>.
## - Müller K, Wickham H (2026). _tibble: Simple Data Frames_.
## doi:10.32614/CRAN.package.tibble
## <https://doi.org/10.32614/CRAN.package.tibble>, R package version 3.3.1,
## <https://CRAN.R-project.org/package=tibble>.
## - Neuwirth E (2022). _RColorBrewer: ColorBrewer Palettes_.
## doi:10.32614/CRAN.package.RColorBrewer
## <https://doi.org/10.32614/CRAN.package.RColorBrewer>, R package version 1.
## 1-3,

```

```

## <https://CRAN.R-project.org/package=RColorBrewer>.
##   - R Core Team (2025). _R: A Language and Environment for Statistical
## Computing_. R Foundation for Statistical Computing, Vienna, Austria.
## <https://www.R-project.org/>.
##   - Schauberger P, Walker A (2025). _openxlsx: Read, Write and Edit xlsx F
##iles_.
## doi:10.32614/CRAN.package.openxlsx
## <https://doi.org/10.32614/CRAN.package.openxlsx>, R package version 4.2.8.
## 1,
## <https://CRAN.R-project.org/package=openxlsx>.
##   - Wickham H (2016). _ggplot2: Elegant Graphics for Data Analysis_.
## Springer-Verlag New York. ISBN 978-3-319-24277-4,
## <https://ggplot2.tidyverse.org>.
##   - Wickham H (2025). _stringr: Simple, Consistent Wrappers for Common Str
##ing
## Operations_. doi:10.32614/CRAN.package.stringr
## <https://doi.org/10.32614/CRAN.package.stringr>, R package version 1.6.0,
## <https://CRAN.R-project.org/package=stringr>.
##   - Wickham H, Bryan J (2025). _readxl: Read Excel Files_.
## doi:10.32614/CRAN.package.readxl
## <https://doi.org/10.32614/CRAN.package.readxl>, R package version 1.4.5,
## <https://CRAN.R-project.org/package=readxl>.
##   - Wickham H, François R, Henry L, Müller K, Vaughan D (2026). _dplyr: A
##Grammar
## of Data Manipulation_. doi:10.32614/CRAN.package.dplyr
## <https://doi.org/10.32614/CRAN.package.dplyr>, R package version 1.2.0,
## <https://CRAN.R-project.org/package=dplyr>.
##   - Wickham H, Henry L (2026). _purrr: Functional Programming Tools_.
## doi:10.32614/CRAN.package.purrr <https://doi.org/10.32614/CRAN.package.pur
##rr>,
## R package version 1.2.1, <https://CRAN.R-project.org/package=purrr>.
##   - Wilkins D (2025). _treemapify: Draw Treemaps in 'ggplot2'_.
## doi:10.32614/CRAN.package.treemapify
## <https://doi.org/10.32614/CRAN.package.treemapify>, R package version 2.6.
## 0,
## <https://CRAN.R-project.org/package=treemapify>.

```